

CHALMERS
UNIVERSITY OF TECHNOLOGY



UNIVERSITY OF GOTHENBURG

Intermediate Representations for LLM-Based Test Generation: A Comparative Study in Automotive Software Verification

Master's Thesis in Computer science and engineering

Bin-Umar Saiid & Oluwatosin Omotoyinbo

Department of Computer Science and Engineering
CHALMERS UNIVERSITY OF TECHNOLOGY
UNIVERSITY OF GOTHENBURG
Gothenburg, Sweden 2026

Master's Thesis 2026

Intermediate Representations for LLM-Based Test Generation: A Comparative Study in Automotive Software Verification

Bin-Umar Saiid & Oluwatosin Omotoyinbo



UNIVERSITY OF
GOTHENBURG



CHALMERS
UNIVERSITY OF TECHNOLOGY

Department of Computer Science and Engineering
Chalmers University of Technology
University of Gothenburg
Gothenburg, Sweden 2026

Intermediate Representations for LLM-Based Test Generation: A Comparative Study in Automotive Software Verification
Bin-Umar Saiid & Oluwatosin Omotoyinbo

© Bin-Umar Saiid & Oluwatosin Omotoyinbo, 2026.

Supervisor: Yinan Yu, Department of Computer Science and Engineering
Examiner: Gregory Gay, Department of Computer Science and Engineering

Master's Thesis 2026
Department of Computer Science and Engineering
Chalmers University of Technology and University of Gothenburg
SE-412 96 Gothenburg
Telephone +46 31 772 1000

Typeset in LaTeX
Gothenburg, Sweden 2026

Bin-Umar Saiid & Oluwatosin Omotoyinbo
Department of Computer Science and Engineering
Chalmers University of Technology and University of Gothenburg

Abstract

Large Language Models (LLMs) can generate test scripts from natural-language requirements, but direct generation often misses boundary behaviors, fault handling, and requirement-specific signal details. We investigate whether intermediate representations (IRs)—artifacts inserted between requirements and executable test scripts—can improve the completeness, traceability, and practical utility of LLM-based test generation in an industrial automotive context.

We design a unified four-path framework: three generation paths (A–C) that vary only the IR type while holding constant the LLM family, discovery infrastructure, and output format—no IR (direct generation), a structured JSON reasoning document, and an executable Python state-machine specification (xSpec)—and one augmentation path (D) that uses the executable IR in a separate enhancement mode to identify gaps in existing human-authored tests rather than generating test scripts from scratch. The framework is evaluated on 31 functional requirements from Volvo Group's body/cabin electronics domain, with three runs per configuration to account for LLM nondeterminism.

Results show that introducing IR improves static test-script quality—as measured by behavioral completeness and requirement traceability—over direct generation. All quality assessments are based on static analysis of test-script content. Comparing IR types, the executable IR outperforms the structured IR on traceability, but the two are statistically indistinguishable on completeness, signal correctness, and value correctness—indicating that structuring the LLM's reasoning provides most of the measurable benefit, with executability adding a targeted advantage. The augmentation path preserves 94.7% of existing human-authored behaviors while identifying coverage gaps with 87.5% precision and 63.2% recall, and a median coverage improvement of +0.302. Most identified gaps (62.5%) are fault-condition behaviors, the category where existing test suites are weakest.

These findings support a staged adoption strategy: structured IRs for immediate quality gains, executable IRs where traceability matters most, and IR-guided augmentation to extend existing trusted tests rather than replace them.

Keywords: LLM, test generation, intermediate representation, automotive, state machine, requirements-based testing, executable specification, gap analysis

Acknowledgements

We would like to express our sincere gratitude to everyone who supported and guided us throughout the course of this thesis.
First and foremost, we extend our deepest thanks to our industrial supervisor, Dhasarathy Parthasarathy, and the core ML team at Volvo Group Truck Technology. Their expert knowledge of the automotive testing domain, willingness to challenge our assumptions, and openness in sharing domain knowledge and tooling were invaluable throughout this work. The depth and rigour of this thesis owe much to their guidance and constant engagement with both the practical and research dimensions of our study.
We would like to thank our academic supervisor, Yinan Yu, for her constructive feedback, thoughtful questions, and steady encouragement throughout the project. Her perspective consistently helped us see our work more clearly and communicate it more precisely.
We are grateful to our examiner, Gregory Gay, for his thorough and rigorous review of this thesis. His detailed comments pushed us to be more careful and more honest about the scope, limitations, and contributions of our findings.
Finally, we owe our deepest thanks to our families and friends for their patience, understanding, and unwavering encouragement throughout this demanding process. This thesis would not have been possible without their support.

Bin-Umar Saiid & Oluwatosin Omotoyinbo, Gothenburg, June 2026

Contents

List of Figures **xiii**

List of Tables **xv**

1 Introduction **1**
1.1 Background and Motivation 1
1.2 Problem Statement 2
1.3 Research Questions 3
1.4 Scope and Delimitations 4
1.5 Significance of the Study 5
1.6 Thesis Structure 6

2 Background **7**
2.1 Automotive Software Testing 7
2.1.1 Test Script Format 8
2.1.2 Test Maintenance Challenges 9
2.2 Requirements Engineering in Automotive 10
2.3 Large Language Models for Code Generation 11
2.3.1 Prompting Strategies 11
2.3.2 Agentic LLMs 12
2.4 Intermediate Representations for Testing 12
2.4.1 Model-Based Testing 12
2.4.2 The IR Formality Spectrum 13
2.5 The Industrial Platform 14
2.5.1 Architecture 14
2.5.2 Data Access Layer 15
2.5.3 Relevance to This Thesis 15

3 Related Work **17**
3.1 LLM-Based Test Generation 17
3.1.1 Boundary Value Analysis and Computational Reasoning . . . 17
3.1.2 Memorization as a Cause of Generalization Failure 18
3.2 Role of Intermediate Representations (IR) 18
3.2.1 Taxonomy of IRs in Recent Literature 19
3.3 Requirement Coverage 19
3.3.1 Fault Detection 19
3.3.2 Semantics-Aware Evaluation 20

3.4 Research Gap 20

4 Methodology **21**
4.1 Research Approach 22
4.1.1 Design Science Research Methodology 22
4.1.2 Experimental Design 23
4.1.3 Quality Dimensions and RQ Mapping 24
4.2 Dataset 25
4.2.1 Source and Context 25
4.2.2 Selection Criteria 25
4.2.3 Dataset Characterization 26
4.3 The Unified Generation Framework 27
4.3.1 PN Script DSL Format 28
4.4 Intermediate Representation Levels 30
4.4.1 Direct-to-TestScript (No-IR) — Path A 30
4.4.2 Structured Plan IR (Path B) 30
4.4.3 Executable Behavioral IR (xSpec) — Path C 31
4.5 Enhancement Mode 33
4.6 Evaluation Design 34
4.6.1 Experimental Procedure 34
4.6.2 Ground Truth and Comparison Basis 35
4.6.3 Metrics 36
4.6.4 Metric Instrument Validation 40
4.6.5 Statistical Analysis 41
4.7 Ethical Considerations 41

5 Results **43**
5.1 Experimental Run Overview 43
5.2 Manual Validation Study 44
5.2.1 BC Extraction 44
5.2.2 Reliability Summary 45
5.3 RQ1: Effect of Intermediate Representation 45
5.3.1 Primary Metrics: Requirement-Centric Evaluation 45
5.3.2 Statistical Comparison (Pooled Analysis) 45
5.3.3 Disaggregated Analysis: Path B vs. Path A and Path C vs. Path A 46
5.3.4 Triangulation: GT-SC against Human-Authored Tests 47
5.4 RQ2: Structured vs. Executable IR 47
5.4.1 Primary Metric Comparison 48
5.4.2 Within-Path Variance: Output Stability Across Runs 49
5.5 RQ3: Augmentation of Existing Tests 49
5.5.1 Path D Dataset 50
5.5.2 Gap Identification: GP and GR 50
5.5.3 Preservation Rate (PR) 50
5.5.4 Coverage Improvement (ΔBC) 52
5.5.5 Reference-Run Sensitivity 52
5.5.6 Qualitative Analysis of Gap and Preservation Types 53

6 Discussion **55**
6.1 Research-Question Interpretation . . . 55
6.1.1 RQ1: Effect of Using an Intermediate Representation . . . 55
6.1.2 RQ2: Structured IR Versus Executable IR . . . 56
6.1.3 RQ3: Augmentation of Existing Human-Authored Tests . . . 57
6.2 Cross-Cutting Interpretation . . . 58
6.2.1 BC Versus GT-SC: Complementary, Not Redundant . . . 58
6.2.2 Instrument-Reliability Effects on Interpretation Confidence . . . 59
6.3 Positioning Against Related Work . . . 60
6.4 Implications for Industrial Adoption . . . 61
6.4.1 Cost and Efficiency . . . 62
6.5 Threats to Validity . . . 63
6.5.1 Construct Validity . . . 63
6.5.2 Internal Validity . . . 63
6.5.3 External Validity . . . 64
6.5.4 Conclusion Validity . . . 64
6.5.5 Reliability and Replicability . . . 64
6.6 Discussion Summary . . . 65

7 Conclusion **67**
7.1 Summary of Findings . . . 67
7.2 Contributions . . . 68
7.3 Future Work . . . 68
7.4 Closing Remarks . . . 69

Bibliography **71**

A Appendix **I**
A.1 Manual Validation Sample . . . I
A.2 Validation Procedure and Worked Example . . . II
A.2.1 Validation Packet Structure . . . II
A.2.2 Worked Example . . . II
A.2.3 Evaluation Tooling . . . III
A.3 RQ3 Per-Handle Enhancement Results . . . IV
A.4 JSON IR Schema . . . V
A.5 xSpec Framework . . . VI

List of Figures

1.1 Research question structure and mapping to generation paths. 5

2.1 Simplified V-model lifecycle used in automotive software development (adapted from ISO 26262 [1]). 8
2.2 Platform architecture overview 15

4.1 Side-by-side overview of the research activities 22
4.2 Unified four-path pipeline 29

5.1 Primary-metric summary across paths (means with SD error bars, from Table 5.4). 46
5.2 RQ2 primary-metric comparison: Path B vs. Path C (means with SD error bars, from Table 5.4). 48
5.3 RQ3 coverage-improvement summary from reported aggregate values and paired-difference counts. 52
5.4 Category distribution of identified gaps, preserved behaviors, and lost behaviors (percentages from Table 5.16). 54

List of Tables

2.1 Comparison of intermediate representation levels. 14

4.1 Mapping of DSRM phases [2] to thesis content. 23
4.2 Dataset filtering pipeline 26
4.3 Dataset complexity distribution. 27
4.4 Discovery agents and responsibilities. 28
4.5 Comparison of IR properties (Paths A–C). 33
4.6 Metrics summary per research question. 39

5.1 Generation completion across the three independent runs. 43
5.2 Human vs. agent BC extraction on 6 validated outputs (one Path C run per requirement). 44
5.3 BC extraction reliability against the 0.85 threshold of Section 4.6.4 (two-rater study, $n = 6$ requirements). 45
5.4 Primary metrics across Paths A, B, C ($n = 31$, means ± SD; per-handle values are first averaged over the three runs). 45
5.5 RQ1 pooled comparison: Paths B and C (averaged per requirement) versus Path A. Wilcoxon signed-rank test on per-requirement paired values; $p_{\text{adj}} = \min(1,\ p \cdot k)$ with $k = 4$ primary metrics. Cliff's δ uses the convention $\delta(\text{IR}, \text{A})$; positive values favor the IR paths. 46
5.6 RQ1 disaggregated comparisons. 47
5.7 GT-SC across Paths A, B, C ($n = 31$ requirements). Mean and median are over per-requirement values. 47
5.8 Path C vs. Path B primary metrics ($n = 31$ paired requirements). . . 48
5.9 Wilcoxon signed-rank tests for Path C vs. Path B ($k = 4$). Bonferroni-adjusted values use $p_{\text{adj}} = \min(1,\ p \cdot k)$; significance is evaluated at $p_{\text{adj}} < 0.05$. 48
5.10 Within-path variance: standard deviation across the three runs, averaged over the 31 requirements (lower indicates more stable generation). 49
5.11 Path D dataset eligibility. "SDK timeout" refers to the Claude Agent SDK's request-timeout limit reached during Phase 2 scoring of the human-authored union; "analyst transient" refers to a single analyst-side read failure on a different handle. 50
5.12 Gap identification: descriptive counts and reliability against the LLM-as-judge second rater. "Genuine gaps (per rater)" is the size of the judge's independent gap set, computed without access to the analyst's classification. 51

5.13 Preservation outcomes at the expected-behavior level. The "Replaced" and "Annotated" outcomes from the original methodology do not apply because Path D as evaluated here does not emit a merged test. . 51
5.14 Behavioral completeness before and after taking the highest-BC Path C run as the enhanced test ($n = 30$). 52
5.15 Reference-run sensitivity for handle `x0400000028DE7EA6`. "Genuine gaps (TP)" uses the same LLM-as-judge second rater applied to the GP/GR step earlier in this section (Table 5.12). 53
5.16 Distribution of identified gaps, preserved behaviors, and lost behaviors by category. Percentages are within column. 53

6.1 Cost and efficiency metrics across IR paths ($n = 93$ runs per path). . 62

A.1 Validation sample: handles, tiers, and expected-behavior counts. . . . I
A.2 Manual-validation audit summary. I
A.3 BC Phase 2 judgements for `x0400000028AA60CA` (worked example, Rater A). III
A.4 Per-handle enhancement results (RQ3). IV

1

Introduction

1.1 Background and Motivation

Automated testing is a cornerstone of modern software development, especially in safety- and quality-critical domains such as automotive systems. Embedded controllers in vehicles are validated through large regression suites of test scripts that are continuously executed on Software-In-the-Loop (SIL) and Hardware-In-the-Loop (HIL) rigs as requirements evolve. Maintaining these suites is labor-intensive and error-prone, particularly when tests are written in a proprietary domain-specific language (DSL) that is tightly coupled to a specific toolchain and execution environment. In practice, engineers must manually interpret natural-language requirements, understand signal interfaces, and then design, implement, and maintain DSL-based test scripts that match the intended behavior of complex stateful functions [3].

Large Language Models (LLMs) have recently emerged as a promising technology for automating parts of this process. Several empirical studies show that LLMs can generate non-trivial unit tests and API tests from source code or natural-language descriptions, often achieving competitive coverage compared to human baselines [4, 5, 6]. However, these studies also highlight important limitations: a significant fraction of generated tests do not compile, fail at runtime, or contain incorrect or incomplete assertions. Moreover, test oracles produced by LLMs may describe the current behavior of an implementation rather than the intended behavior from the requirements, which risks giving a false sense of quality [7]. A survey work on LLM-based test generation therefore emphasizes the need for guardrails, intermediate artifacts, and validation loops, rather than relying on one-shot prompts that directly map requirements to executable tests [8].

At the same time, industrial organizations already possess extensive suites of engineer-authored test scripts that encode domain knowledge, implicit conventions, and years of debugging experience. These scripts are socially and organizationally trusted: they have been reviewed, executed on rigs, and used as arguments in safety cases. Simply discarding them and replacing everything with “black-box” LLM-generated tests is neither realistic nor desirable. Industrial stakeholders want to exploit LLMs to reduce routine work and improve coverage without losing the value embedded in existing artifacts.

This tension motivates a central hypothesis: intermediate representations (IRs) inserted between natural-language requirements and executable test scripts can make LLM-based generation more accurate, auditable, and controllable. More specifically, IRs of different formality levels—from structured but non-executable reasoning documents to fully executable, validated state-machine models—may offer different trade-offs between cost and quality. Furthermore, the same IR-based mechanism may enable augmentation of existing human-authored tests rather than wholesale replacement.

1.2 Problem Statement

This thesis is motivated by the practical needs of Volvo Group that maintains an LLM-based workflow for generating test scripts from natural-language requirements and signal metadata. In this workflow, an executable state-machine specification (referred to as xSpec) written in Python serves as a bridge between natural language and executable test scripts. This thesis does not design or modify the xSpec mechanism itself. Instead, it evaluates the pre-existing workflow and compares it against alternative generation pipelines.

This thesis is based directly on exploratory work conducted in an industrial project course in the same organization. That earlier study investigated two candidate IR representations for this generation task: Python state machines (the xSpec approach already used in the workflow) and Linear Temporal Logic (LTL). The evaluation was conducted on a small number of requirements using a different technical environment —DSPy signatures with Cline as the orchestrator rather than the Anthropic agentic workflow used here. The preliminary findings suggested that Python was more tractable than LTL for generating automotive test scripts in this domain: LTL's formal specification style created a gap between the verified specification and the executable DSL output that was difficult to bridge in practice. Those findings motivated the choice of xSpec as the executable IR in the present study.

Despite the perceived benefits of this intermediate step, three interrelated problems remain unresolved:

Accuracy gap:
When an LLM generates test scripts directly from natural-language requirements without an intermediate representation, the resulting scripts frequently miss behavioral nuances such as boundary conditions, timing constraints, and fault-handling scenarios. Existing research suggests a related pattern: while LLMs excel at semantic comprehension, they struggle at systematic boundary analysis and edge-case enumeration [9]. It is unclear whether and how much an IR-guided approach mitigates this gap in an industrial automotive setting.

Formality gap:
Recent work has begun to explore multi-step pipelines and IRs to improve controllability. For example, generating structured test plans or Gherkin scenarios as

an intermediate bridge, or using LLM-generated control-flow graphs as a JSON IR from which tests are deterministically derived [10]. However, these studies typically evaluate a single IR type against a no-IR baseline. We found limited evidence of controlled same-dataset comparisons across multiple IR formality levels for automotive DSL test generation—from a non-executable structured representation (e.g., a JSON reasoning document) to a fully executable, validated model (e.g., a Python state machine). It is therefore unknown whether the benefit of an intermediate step comes specifically from executability and validation, or whether the general act of structuring the LLM's reasoning is sufficient.

Practicality gap:
Most LLM-based approaches either generate entirely new tests or entirely new models [4, 5]. They rarely address the problem of augmenting existing, trusted test scripts in a controlled way. In model-based testing (MBT), state-based or Simulink models have a long tradition in automotive software for systematically deriving test cases [3], but these require manual model construction and do not leverage existing test suites. In practice, organizations care deeply about preserving and improving what they already have—not only about producing entirely new artifacts. A mechanism that can detect gaps in existing tests through IR-based comparison, and then fill those gaps while preserving human-authored content, would directly address this industrial need.

In summary, this thesis tackles a practical question faced by many industrial teams: instead of asking an LLM to "magically" write good tests from requirements, can we use intermediate representations to systematically guide and check the generation process, so that the resulting test scripts are more complete, more trustworthy, and able to extend existing human-written tests rather than replace them?

1.3 Research Questions

Building on the problem statement above, we formulate three research questions. Together, they investigate the role of intermediate representation executability and validation in LLM-based test generation and the feasibility of IR-guided test augmentation. We distinguish between two classes of intermediate representation:
Structured IRs, which capture requirement semantics in a machine-readable format—in this thesis, a JSON document validated against a schema.
Executable IRs, which encode semantics in a format that can be validated through execution as in this thesis, verified Python state machines. A structured IR captures *what* the requirement specifies without execution, while an executable IR adds the capability to verify that specification against itself.

RQ1. Does using an intermediate representation improve LLM-based test generation from natural-language requirements compared to direct generation?

This question compares a direct approach—where the LLM generates test scripts

solely from requirements and signal metadata—with IR-based approaches that introduce a structured intermediate artifact before test script generation. To assess the aggregate effect of structured intermediate reasoning, we pool both structured and executable IR paths and compare them against the direct-generation. We measure the requirement-centric primary metrics defined in Chapter 4 (Behavioral Completeness, Requirement Traceability, Signal Correctness, and Value Correctness) and triangulate against human-authored tests using a ground-truth scenario-coverage (GT-SC) comparison.

RQ2. Does an executable IR produce better test designs than a structured IR?

We compare generation guided by a structured IR (reasoning.json with schema validation) against generation guided by an executable IR (xspec.py with scenario validation). Both IRs capture the same requirement information (signals, behaviors, modes), but differ in whether they can verify internal consistency through execution. This question isolates the contribution of executability and self-verification (through ScenarioRunner validation) from the general benefit of structuring the generation process. Superior test plans from the executable IR would indicate that the ability to validate requirement decomposition against itself improves test design quality.

RQ3. Can an executable IR support gap analysis of existing human-authored tests?

Industrial practice rarely starts from a blank slate: many requirements already have associated, trusted test scripts. RQ3 therefore evaluates an enhancement mode in which an executable IR (xSpec) is generated from a requirement, compared against the behavior encoded in the existing human-authored test, producing a structured report that identifies behavioral gaps in the existing test and flags which of its existing behaviors are preserved or lost relative to the candidate. We measure (i) the precision and recall of the automatically detected gaps, (ii) the proportion of human-authored expected behaviors preserved by the candidate test, and (iii) the net improvement in requirement-level scenario coverage that filling the gaps would yield, relative to the existing human-authored test.

1.4 Scope and Delimitations

This work focuses on system-level functional test generation for automotive embedded software, specifically targeting one DSL-based test format used for HIL and SIL verification. The evaluation dataset is drawn from a single vehicle platform and functional domain within a commercial vehicle manufacturer. While this industrial grounding provides ecological validity, it also constrains generalisability:

- The test format is a proprietary DSL; results may not transfer directly to other test languages or frameworks.

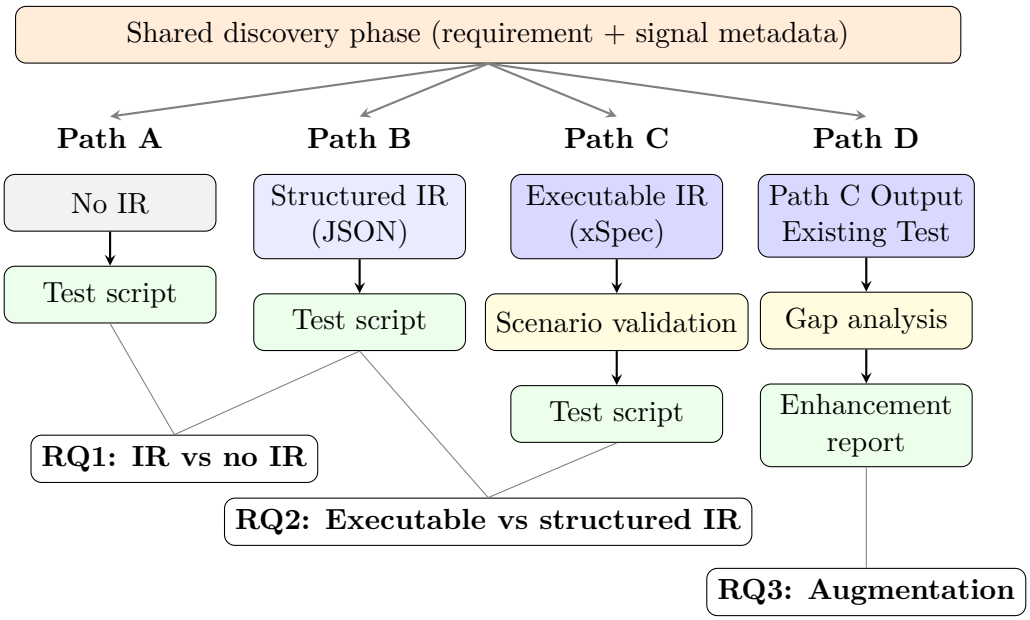


Figure 1.1: Research question structure and mapping to generation paths.

- All experiments use a single LLM (Claude, Anthropic); findings may differ with other models or configurations.
- The evaluation covers one functional domain (level control); cross-domain generalisability is not assessed.
- We evaluate test quality through static analysis metrics (scenario coverage, signal accuracy, behavioral completeness) rather than dynamic execution on SIL/HIL rigs, which remains future work.

1.5 Significance of the Study

The findings of this thesis carry both practical and scientific significance. Its central practical message is one of *augmentation rather than replacement*. Instead of treating LLM-based generation as a way to discard and regenerate test suites from scratch, this work shows that the same IR machinery can be turned inward to *extend* the tests an organization already trusts. This matters because, in safety-critical automotive verification, existing test scripts are not disposable. They encode years of domain knowledge, have been executed on rigs, and are cited in safety cases. Any credible adoption path must add to that asset base rather than sweep it aside, and the results here show that an executable IR can do exactly that.

For practitioners, the results offer an evidence-based, staged path to adopting LLM-based test generation without abandoning existing quality controls. Inserting an intermediate representation between requirements and test scripts measurably improves the behavioral completeness and requirement traceability of generated tests over direct generation, and a lightweight structured IR already captures most

of this benefit, lowering the barrier to entry for teams that cannot invest in heavier tooling. Most importantly, the augmentation mode (Path D) extends trusted, human-authored tests rather than replacing them: it preserves the large majority of existing behaviors while surfacing only the coverage gaps, most of them fault-condition behaviors where existing suites are weakest. This makes the approach deployable as a reviewer aid within current verification workflows rather than a risky wholesale replacement.

For researchers, the thesis contributes a controlled methodology that isolates the effect of IR *formality*: a unified framework that varies only the intermediate representation while holding the LLM family, discovery infrastructure, and output format constant. This separates two effects that prior work tends to conflate, the benefit of *structuring* the model's reasoning and the benefit of *executability*. It also reframes the problem from pure generation to gap-directed augmentation of trusted artifacts, contributing an IR-based method, augmentation-specific metrics (gap precision and recall, preservation rate, and coverage delta), and an evaluation grounded in a real industrial dataset rather than a synthetic benchmark.

1.6 Thesis Structure

The remainder of this thesis is structured as follows.

Chapter 2 provides the background necessary to understand this work, including automotive software testing, requirements engineering practices, LLMs for code generation, intermediate representations in testing, and the industrial platform within which the implementation is deployed.

Chapter 3 reviews related work on LLM-based test generation, intermediate representations in AI-assisted generation, automotive model-based testing, and requirement coverage evaluation, culminating in a characterisation of the research gap.

Chapter 4 presents the methodology: the research approach, the design of the four generation paths, the intermediate representations, and the evaluation design including dataset and metrics.

Chapter 5 reports the results of the comparative evaluation across all three research questions.

Chapter 6 discusses the implications for industrial adoption, threats to validity, and positioning relative to related work.

Finally, Chapter 7 concludes the thesis and outlines directions for future work.

2

Background

This chapter introduces the concepts, technologies, and industrial context necessary to understand the remainder of the thesis. We begin with automotive software testing and the role of test scripts in verification in Section 2.1, then describe how requirements are managed and structured in the automotive domain in Section 2.2. Section 2.3 provides a brief overview of Large Language Models and agentic code generation. Section 2.4 discusses intermediate representations in the context of test generation, covering both traditional model-based approaches and the emerging idea of LLM-friendly executable specifications. Finally, Section 2.5 introduces the platform on which this work is built.

2.1 Automotive Software Testing

Modern commercial vehicles contain dozens of electronic control units (ECUs) that collectively implement safety-critical and comfort functions ranging from braking and steering to lighting and suspension control. The software on these ECUs is developed following the V-model, a lifecycle framework widely adopted in automotive engineering and referenced by the ISO 26262 functional safety standard [1]. In the V-model (Figure 2.1), verification activities on the right-hand side mirror the design activities on the left: unitary tests verify individual software modules, integration tests verify component interactions, and system-level tests verify that the implemented behavior matches the natural-language requirements from which the software was specified.

System-level verification is typically performed on two types of test rigs. Hardware-In-the-Loop (HIL) rigs embed the physical ECU in a simulated vehicle environment: real CAN/LIN bus communication, simulated sensor inputs, and real-time execution allow engineers to verify timing-sensitive behavior under conditions close to production. Software-In-the-Loop (SIL) rigs replace the physical ECU with the compiled application software running on a host PC, enabling faster iteration and wider accessibility at the cost of lower fidelity for timing and electrical behavior. In both cases, the test rig executes a test script—a sequential program that stimulates the ECU with signal values, waits for processing, and then checks that output signals match the expected behavior defined in the requirement.

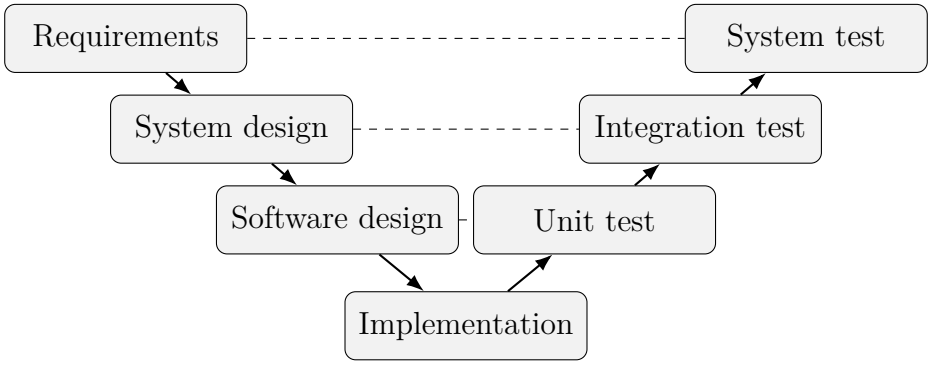


Figure 2.1: Simplified V-model lifecycle used in automotive software development (adapted from ISO 26262 [1]).

2.1.1 Test Script Format

In the industrial context of this thesis, test scripts are written in a proprietary domain-specific language (DSL) called PN Script, executed by a dedicated test runner. Because the DSL is proprietary to Volvo Group, its full grammar cannot be disclosed; we describe its structure and command vocabulary below and provide a representative code excerpt in Listing 2.1. A test script is structured as a sequence of named sections:

- *Preparation Sequence* that declares variables and configures loop counters.
- *Main Sequence* that contains the core test logic: precondition setup, stimulus-response steps, and assertions.
- *Completion Sequence* for teardown and cleanup.
- *Conditions Sequence* and *Constraints Sequence* for execution guards and post-conditions.

Within these sections, the test author composes commands from a fixed vocabulary. The most important commands are:

- **Set** — assigns a value to a signal on a specific communication bus, targeting a named node, frame, and signal. For example, a `Set` command might place a CAN signal representing a switch request into the "pressed" state.
- **Check** — verifies that a signal has reached an expected value. Comparators include equality, inequality, and range-based checks. Failed checks cause the test to report a deviation.
- **Wait** — pauses execution for a specified duration, allowing the ECU to process inputs and propagate outputs. Timing durations depend on the signal type: value changes typically require 200–400 ms propagation time, while edge-triggered events may require longer windows.

- **AwaitValueMatch** — a conditional wait that blocks until one or more signal conditions are satisfied or a timeout expires, combining the semantics of `Wait` and `Check`.
- **ParameterControl** — configures internal ECU parameters (identified by diagnostic object identifiers, or DOIDs) via diagnostic services, enabling or disabling features that affect the requirement under test.
- **ForLoop** — iterates over a range of values, commonly used to sweep through vehicle operating modes (e.g., modes 2 through 6) and verify that behavior is consistent across modes.

A typical test case follows a pattern: set preconditions (vehicle mode, feature-enable parameters), apply a stimulus (set an input signal), wait for the ECU to respond, and then check one or more output signals against expected values. This pattern is repeated for each behavior specified in the requirement, often across multiple vehicle operating modes via loop constructs. Listing 2.1 shows a concrete example of this pattern for a battery temperature selection requirement.

Listing 2.1: PN Script excerpt: one test case for Battery Temperature Selection. The test sets two LIN temperature inputs and checks the CAN output.

```
Comment:
;Case 1 (b1): Both valid -- Temp1 > Temp2 => BatteryTemp =
   MAX(T1,T2) = 30 degC
StateChange:
;Set LIN_BatteryTemp1_1=75 (30 degC), LIN_BatteryTemp2_1=55
   (10 degC)
Set(Type(LIN) Node(BatS2) Message(Bats2_toCCIOM_FR1_L11)
    Signal(LIN_BatteryTemp1_1) Value(75))
Set(Type(LIN) Node(BatS2) Message(Bats2_toCCIOM_FR1_L11)
    Signal(LIN_BatteryTemp2_1) Value(55))
Wait(2000 ms)
Check(Type(CAN) Node(CCIOM) Message(CCIOM_02P_C04)
      Signal(BatteryTemp) Comparer(=) Value(30))
```

2.1.2 Test Maintenance Challenges

Several properties of this format make manual test authoring and maintenance burdensome. First, signal references are verbose: each `Set` or `Check` command must specify the bus type, node, frame, and signal name, all of which can change across vehicle platform variants. Second, the mapping from a natural-language behavior description ("when the switch is pressed, the body shall rise") to the correct sequence of `Set`, `Wait`, and `Check` commands requires detailed knowledge of signal interfaces, timing constraints, and ECU-internal parameter configurations. Third, because tests are written in a DSL rather than a general-purpose programming language, standard code-analysis tooling (linters, type checkers, refactoring tools) is not available. Engineers rely on manual review and rig execution to catch errors.

These properties make the format a natural target for LLM-based automation: the structure is regular enough to be learnable from examples, the signal vocabulary is finite and discoverable from system metadata, and the manual effort is concentrated in the repetitive assembly of `Set`/`Wait`/`Check` patterns rather than in algorithmic complexity.

2.2 Requirements Engineering in Automotive

The requirements that drive test script authoring are managed in SystemWeaver, a commercial model-based systems engineering (MBSE) tool that organizes the vehicle's electrical/electronic (E/E) architecture as a hierarchy of items. At the top level, vehicle variants — corresponding to different truck platforms — decompose into functional domains, which in turn contain Logical Design Components (LDCs). An LDC is the organization's term for a component grouping related functionality; for example, all level-control logic for the vehicle body may be encapsulated within a single LDC. Individual functional requirements are then attached at this level, forming the traceable basis for test script development.

A functional requirement specifies the expected behavior of a software function in terms of input signals, output signals, preconditions, and expected outputs. Input and output signals are typed and bound to specific communication buses (CAN or LIN), message frames, and nodes. The requirement text is written in natural language, often in a semi-structured "when . . . shall . . . " format, supplemented with signal tables and condition matrices. For example, a simplified requirement might read: "*When both battery temperature sensors report valid readings, the CCIOM shall set BatteryTemp to the maximum of the two sensor values. When a sensor reports an error, the CCIOM shall use the last valid reading for up to 10 minutes, after which it shall fall back to a default value of 20 °C.*" Real requirements follow this pattern but include additional detail such as raw-to-physical conversion formulas, timing tolerances, and mode-dependent conditions.

Traceability between requirements and test cases is maintained through explicit links in the engineering tool: a requirement item references one or more test case items, each of which contains the DSL test script as an attribute. This traceability chain—requirement to test case to execution result—is a cornerstone of the V-model and is required for safety argumentation under ISO 26262 [1]. In the dataset used in this thesis, most requirements are linked to one or two test cases (see Chapter 4 for details), reflecting a common industrial pattern of one nominal-path test and one boundary or off-nominal test per requirement.

An important characteristic of this requirements ecosystem is that signal metadata—names, default values, enumeration labels, bus assignments, and frame identifiers—is distributed across multiple data sources: the requirements management tool, communication database files, and diagnostic parameter registries. Resolving the complete signal context for a single requirement often requires querying several of these sources and reconciling naming conventions (e.g., dot-separated names in the re-

quirements tool versus underscore-separated names in bus databases). This signal-resolution complexity is one reason why automated test generation benefits from a dedicated information-gathering phase.

2.3 Large Language Models for Code Generation

Large Language Models (LLMs) are neural networks based on the Transformer architecture [11] that are pretrained on large corpora of text and code to predict the next token in a sequence, and subsequently fine-tuned to follow instructions [12]. Models such as GPT-4 [13], Claude [14] and Gemini [15] have demonstrated strong capabilities in code generation, including the ability to produce syntactically correct programs from natural-language descriptions and to adapt to unfamiliar APIs and formats through in-context learning.

2.3.1 Prompting Strategies

The effectiveness of LLMs for code generation depends heavily on how the task is communicated. Several prompting strategies have emerged:

- **Zero-shot prompting** provides only the task description without examples [16]. This tests the model's ability to generalize from its training data to a novel format or domain. For instance, the prompt might state: "*Generate a PN Script test case for the following requirement: ...*" without providing any sample test script.

- **Few-shot prompting** includes one or more input–output examples in the prompt, allowing the model to infer structural patterns and conventions from the examples [16]. For example, the prompt might include a complete requirement–test-script pair for one function, followed by a new requirement for which the model must produce a test. For domain-specific formats, even a single well-chosen example can substantially improve output quality.

- **Chain-of-Thought (CoT) prompting** instructs the model to produce intermediate reasoning steps before generating the final output [17]. For example, the prompt might instruct: "*First, list the behaviors specified in the requirement. Then, identify the input and output signals for each behavior. Finally, compose the test script.*" This decomposition reduces the chance of omitting behaviors or misassigning signals.

- **Structured output prompting** constrains the model to produce output in a specific format—for example, JSON, YAML, or Python—by specifying the schema or template in the prompt. For instance, requiring the model to emit a JSON object with fields `behaviors`, `signals`, and `test_steps` before generating the final script. This reduces formatting errors and enables downstream validation.

2.3.2 Agentic LLMs

A more recent development is the use of Large Language Models (LLMs) as agents: autonomous systems capable of planning multi-step actions, invoking external tools, and iterating based on environmental feedback [18, 19]. Rather than generating a complete output through a single prompt, an agentic LLM decomposes a complex task into manageable sub-goals, executes these via tool calls (e.g., querying databases, executing scripts, or reading files), and synthesizes the results. This architecture is particularly effective for automated test generation, where the system must:

- Retrieve requirement details and signal metadata from engineering tools.
- Optionally generate and validate a structured intermediate representation (e.g., a decision table or control flow graph).
- Synthesize the final executable test script using the aggregated information.

Sub-agent orchestration extends this paradigm by employing a coordinating agent to delegate specialized sub-tasks to auxiliary agents with distinct capabilities or model configurations. For instance, a lightweight model may be utilized for routine metadata lookups to preserve context, while a more capable model manages steps requiring complex reasoning and script synthesis. This hierarchical architecture is the one the implementation adopts, as described in Chapter 4.

2.4 Intermediate Representations for Testing

An intermediate representation (IR), in the context of this thesis, is any structured artifact produced between the natural-language requirement and the final test script. The role of the IR is to make the LLM's interpretation of the requirement explicit, inspectable, and—in the case of an executable IR—executable and verifiable before the final code generation step [20, 21].

2.4.1 Model-Based Testing

The use of IRs for test derivation has a long history in model-based testing (MBT). In MBT, a model of the system under test—typically a finite state machine, a statechart, or a Simulink model—serves as the basis for systematic test case generation. Coverage criteria such as state coverage, transition coverage, and Modified Condition/Decision Coverage (MC/DC) guide the selection of test cases from the model's state space [22]. In automotive software, MBT has been applied to derive test cases from Simulink models and state-based specifications to handle the increasing complexity of electronic control units [23, 3].

However, traditional MBT relies on manually constructed models that are expensive to create and maintain, and that require specialist modeling skills. Moreover, the modeling notations used in MBT—such as UML statecharts, Simulink, or pro-

prietary modeling languages—present a "syntax cliff" for Large Language Models. Such models are often under-represented in LLM training datasets compared to general-purpose programming languages [24, 25]. Recent empirical work confirms this gap: while LLMs can achieve acceptable *syntactic* validity for modeling languages—exceeding 90% for PlantUML-formatted SysML when given format specifications [26]—*semantic* accuracy remains substantially lower, with state-diagram generation reaching only approximately 50% F1 on semantic correctness [26] and SysML v2 generation achieving roughly 62% semantic precision [27]. Iterative, agent-based correction loops can push syntactic validity to near 100% [28], but they add pipeline complexity and do not resolve the underlying semantic gap. Simulink presents an even harder target: its graphical block-diagram format is poorly suited to text-based generation, and current LLM-based Simulink assistants remain at a prototypical stage [29].

2.4.2 The IR Formality Spectrum

Between the extremes of no IR (direct generation) and a full formal model (MBT), a spectrum of intermediate representations exists. Synthesizing concepts from chain-of-thought reasoning [17], structured phase decoupling [21], and executable model-based testing [22], we distinguish three levels of formality that are relevant to this thesis:

No IR (direct generation). The LLM receives the requirement and signal metadata and produces the test script in a single generation step. There is no explicit intermediate artifact and no validation checkpoint. This is the simplest approach but offers no opportunity to catch reasoning errors before they manifest in the final output.

Structured IR. The LLM produces a structured but non-executable document—for example, a JSON object that enumerates identified behaviors, lists input and output signals with their possible values, and sketches a coverage plan. This document can be validated for structural correctness (e.g., JSON schema conformance) and inspected by a human reviewer, but it cannot be executed or verified against the requirement's logic. The structured IR serves as a "structured scratchpad" that encourages the LLM to decompose the requirement before generating test script. Listing 4.1 in Chapter 4 shows a concrete example of such a structured behavior entry.

Executable IR. The LLM produces an executable specification—in our case, a Python state machine with declared signals, states, transitions, and behaviors. This specification includes a set of scenarios that exercise each declared behavior and can be executed to verify internal consistency: if a scenario fails, the specification contains a logical error that would propagate to the generated test script. The executable IR thus introduces a mandatory validation checkpoint between requirement interpretation and code generation. Listing 4.2 in Chapter 4 shows a representative excerpt of such a specification.

The central empirical question we address is where on this spectrum the quality–cost trade-off is most favorable for industrial automotive test generation.

Table 2.1: Comparison of intermediate representation levels.

Property	No IR (Path A)	Structured IR (Path B)	Executable IR (Path C/D)
Artifact	None	JSON reasoning document	Python state machine (xSpec)
Executable	–	No	Yes
Validation method	None	JSON schema conformance	Scenario execution
State-machine semantics	Not represented	Not represented	Explicit (states, transitions)
Boundary detection	Implicit in LLM output	Listed in coverage plan	Encoded as scenario assertions
Timing logic	Implicit	Descriptive	Executable guards
Auditability	Final output only	IR + final output	IR + execution log + final output
Error checkpoint	None	Structural only	Structural + behavioral

2.5 The Industrial Platform

The work in this thesis is implemented within an industrial LLM platform built on Anthropic's Claude model family [14] that follows a plugin-based architecture. The terminology used below (*skills*, *agents*, *plugins*) reflects the platform's own abstractions. This section describes the platform's structure at a level sufficient to understand the implementation described in Chapter 4.

2.5.1 Architecture

The platform is organized around three abstractions:

- **Skills** are user-facing capabilities, invoked via a command interface with named parameters. A skill defines the high-level workflow: what information to gather, what artifacts to produce, and in what order.
- **Agents** are specialized sub-components that perform a single focused task like discovering signals for a requirement, generating an executable specification, or parsing an existing test script. Agents have access to a controlled set of tools (file system operations, external API calls) and can be configured with different LLM model tiers based on task complexity.
- **Plugins** package skills, agents, and their configurations into deployable units. The platform's marketplace allows teams to install and configure plugins for their specific needs.

In this architecture, a skill invocation triggers a multi-agent workflow: the skill's orchestrating prompt determines the execution order, and each agent is invoked as a sub-task with its own system prompt, tool access, and model selection. This allows the pipeline to combine lightweight models (for data retrieval tasks) with more capable models (for complex reasoning and generation tasks). Throughout this thesis, *lightweight* denotes a smaller, faster, and cheaper model tier suited to high-volume lookup and enumeration tasks, while *capable* denotes a larger model tier with stronger multi-step reasoning used for analysis and code generation. The

concrete tier-to-model assignment used in this work (Claude Haiku, Sonnet, and Opus, respectively) is specified in Section 4.6.1.

2.5.2 Data Access Layer

The platform provides access to engineering data through a Model Context Protocol (MCP) server that acts as a unified API layer over multiple backend systems. Through this server, agents can query:

- The requirements management system, to retrieve requirement descriptions, signal references, and architectural metadata.
- Communication databases, to resolve signal names to their bus-level definitions (bus type, frame, transmitter node, enumeration values).
- Diagnostic parameter registries, to discover configurable parameters (DOIDs) and their valid value ranges.

Additionally, a pre-exported snapshot of the requirements hierarchy is available as a local file index, enabling agents to navigate the component tree and resolve parent-child relationships without incurring network round-trips for every query. This hybrid approach—local index for structural navigation, remote API for detailed lookups—balances query latency against data freshness. Figure 2.2 provides an overview of the platform architecture and data flow.

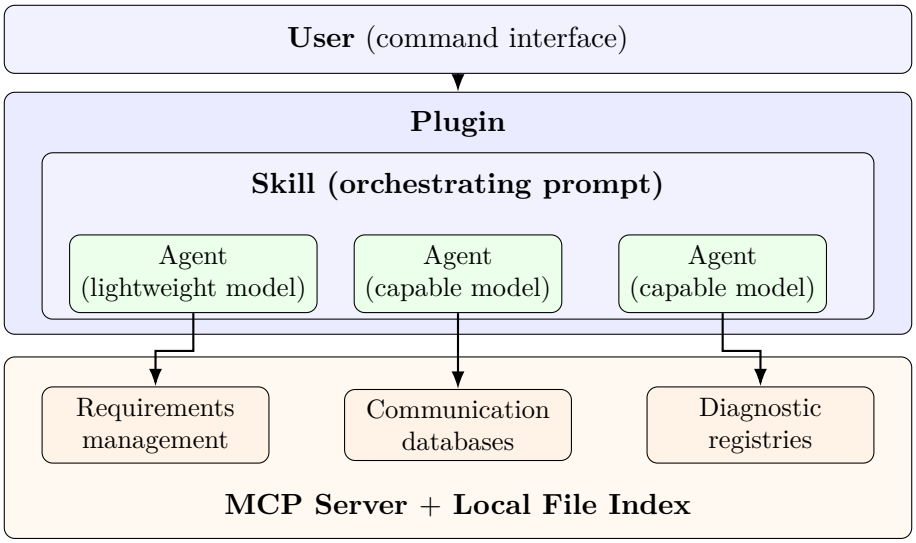


Figure 2.2: Platform architecture overview

2.5.3 Relevance to This Thesis

The platform's architecture is directly relevant to the experimental design of this thesis. Because all four generation paths (no IR, representative IR, executable IR, enhancement) are realized as configurations of a single skill, they share the same data-access infrastructure, the same discovery agents, and the same signal-resolution

logic. This eliminates a class of confounding variables: any differences in output quality between paths can be attributed to the IR type rather than to differences in input data or tooling.

With this technical and industrial context established, the next chapter reviews prior work and positions the specific research gaps addressed in this work.

3

Related Work

The automation of Requirements-Based Test Generation (RBTG) has evolved from a niche application of rule-based heuristics to a central challenge in the era of Generative AI. This chapter critically reviews the evolution of this field, categorizing the literature into three streams:

1. The limitations of direct generation, specifically regarding computational reasoning and data contamination;

2. The strategic role of Intermediate Representations (IR) in constraining these failures;

3. The emerging shift in evaluation objectives from structural code coverage to semantic requirement coverage.

3.1 LLM-Based Test Generation

The integration of Large Language Models (LLMs) into software testing has shifted the paradigm from syntactic translation to semantic synthesis. Early approaches utilized Seq2Seq models or fine-tuned BERT variants to map natural-language descriptions to code segments. However, the advent of general-purpose transformers (e.g. GPT-4, Llama 3) introduced *direct generation*: prompting a model to synthesize entire test scripts directly from requirements. While promising, this approach is hindered by two fundamental limitations: the "computational dwarf" paradox and the causal role of memorization in performance degradation.

3.1.1 Boundary Value Analysis and Computational Reasoning

Recent empirical studies reveal a critical "reasoning gap" when LLMs are applied to complex logic. The GBCV (Generated Benchmark from Control-Flow) framework demonstrates that while LLMs excel at parsing linguistic intent ("semantic giants"), they frequently fail at rigorous computational reasoning ("computational dwarfs") [30]. This deficit is most acute in Boundary Value Analysis (BVA).

BVA is a black-box testing technique based on the principle that errors are most likely to occur at the boundaries of input domains rather than in the centre. For example, for a requirement specifying an integer input range "$10 \leq x \leq 100$", rigorous BVA dictates testing the valid boundaries (10, 100) and the invalid adjacent values (9, 101). Ma et al. found that LLMs struggle to mathematically calculate the precise input values required to trigger these boundary conditions in nested logic, often hallucinating plausible-looking but functionally ineffective inputs that fail to exercise the edge cases [30].

3.1.2 Memorization as a Cause of Generalization Failure

A significant threat to the validity of LLM-based testing is data contamination, where the model's apparent reasoning capability is actually a retrieval of memorized training data. This was rigorously quantified by the ULT and PLT benchmarks [31].

- **ULT** (Unit Level Testing) consists of public code likely present in the LLM's training corpus.
- **PLT** (Private Level Testing) consists of novel, time-stamped code guaranteed to be unseen.

Results showed a precipitous drop in performance on PLT compared to ULT [31]. This confirms that high success rates on public leaderboards are largely driven by the model recalling memorized solution patterns rather than reasoning about novel requirements. For industrial applications—where requirements are proprietary, unique, and evolving—direct generation techniques are therefore prone to failure, necessitating a more robust, structure-aware approach that prevents the model from relying on memorized priors.

3.2 Role of Intermediate Representations (IR)

Direct generation forces the LLM to simultaneously manage linguistic parsing, test logic design, and syntactic correctness, thereby increasing the risk of hallucination and logic drift. In response, contemporary frameworks have increasingly adopted Intermediate Representations (IRs). The literature suggests that an IR acts as a necessary "cognitive scaffold", decomposing the complex generation task into two manageable sub-problems: *interpretation* (natural language $\rightarrow$ IR) and *implementation* (IR $\rightarrow$ code).

An IR reduces the risks of direct generation by:

- **Constraining the search space:** By forcing the model first to output a structured schema (such as a state machine or logic formula), the system restricts the model's creative freedom, preventing it from hallucinating features that do not exist in the textual requirement [32, 33].
- **Decoupling logic from syntax:** An IR allows the system to validate the log-

ical correctness of the test design (e.g. “Are all states covered?”) before dealing with the syntactic complexity of the target testing language (e.g. Python, CAPL, or TTCN-3) [33, 32].

3.2.1 Taxonomy of IRs in Recent Literature

Visual Models (SysML/UML) The SysMBench framework evaluates the generation of SysML v2 models [27]. While valuable for architecture, these models face a “syntax cliff”: LLMs struggle to generate the verbose, strict syntax of graphical modeling languages without extensive fine-tuning, achieving only approximately 62% semantic precision [27].

Formal Logic (CTL/B-Method). For safety-critical systems, datasets like Natural2CTL [34] and benchmarks like CLEVER [35] focus on translating requirements into Computation Tree Logic (CTL) or B-Method specifications. While these provide mathematical guarantees (verified via solvers like Z3), they often suffer from an “executability gap”—they prove the design is safe but do not natively generate the executable scripts needed for Hardware-in-the-Loop (HIL) validation.

Executable Skeletons. The most relevant stream for this thesis involves lightweight, executable IRs. Han et al. propose TriGen, which utilizes a “semantic skeleton” (Subject–Verb–Object–Polarity) to validate test procedures, reducing hallucination rates to 0.183 [32]. Similarly, Liao et al. introduce JUnitGenie, which uses Jimple (a three-address code IR) and Control Flow Graphs (CFG) to perform path-sensitive generation, improving branch coverage by approximately 30% over baselines that skip the IR step [33].

3.3 Requirement Coverage

A critical evolution in the 2024–2026 literature is the shift in evaluation objectives. Traditional metrics like code coverage (statement/branch) are increasingly viewed as insufficient for validation because they cannot detect “features of omission”—logic specified in the requirement but missing from the implementation.

3.3.1 Fault Detection

The RTM (Requirements coverage-guided Test suite Minimization) framework challenges the industry standard of minimizing test suites based solely on code execution [36]. Their findings introduce the concept of “semantic redundancy”. In safety-critical domains (like automotive), having multiple test cases that cover the same code path but represent different semantic scenarios is not waste; it is robustness. Pan et al. demonstrate that optimizing for requirement coverage (ensuring every functional constraint is tested) yields a significantly higher fault detection rate (FDR) than optimizing for code coverage alone [36].

3.3.2 Semantics-Aware Evaluation

Finally, the definition of "quality" has moved beyond text similarity metrics. New benchmarks employ semantics-aware metrics. For instance, SysMEval decomposes models into "atomic semantic claims" to verify structural facts [27], while Doc2Spec uses grammar induction to verify that generated specifications adhere to the strict rules of the domain [37]. This aligns with the objectives of this thesis: to validate the behavioral logic of the test case, not just its textual resemblance to a reference script.

3.4 Research Gap

While the literature establishes that IR-guided generation significantly outperforms direct generation in terms of traceability and correctness [32, 33], the existing approaches are polarized. They are either too abstract (SysML) for direct execution [27] or too theoretical (B-Method) for rapid industrial iteration [35]. There remains a paucity of research on lightweight, signal-centric intermediate representations specifically designed for automotive test orchestration. This thesis addresses this gap by introducing xSpec, an executable IR that serves both as a validation layer (checking logic against requirements) and as a generation source (producing vendor-specific scripts from the prototype), thereby directly targeting the maximization of requirement coverage.

4

Methodology

This chapter describes the research approach, the dataset, the design of the four generation paths, and the evaluation framework. The main research activities and their contributions to the research questions are:

1. **Problem analysis and research design** (Section 4.1). We define the DSRM process, formulate the three research questions, and design a within-subjects controlled comparison isolating the IR type as the independent variable.

2. **Dataset construction** (Section 4.2). We select and characterize 31 industrial requirements from Volvo Group's CCIOM domain, stratified by complexity, to serve as the shared evaluation corpus for all paths.

3. **Framework and path implementation** (Sections 4.3–4.4). We implement the unified generation framework with three IR levels—no IR (Path A), structured JSON IR (Path B), and executable Python IR (Path C)—sharing the same discovery and signal-resolution infrastructure. This directly enables **RQ1** and **RQ2**.

4. **Enhancement mode** (Section 4.5). We implement Path D, which uses the executable IR to identify coverage gaps in existing human-authored tests and generate augmentation patches. This enables **RQ3**.

5. **Evaluation design and execution** (Section 4.6; Chapters 5–6). We define six primary and secondary metrics, establish a ground-truth strategy, and apply Wilcoxon signed-rank tests with Bonferroni correction to answer **RQ1**–**RQ3**.

Figure 4.1 summarizes these research activities visually: it places the four generation paths side by side and shows, for each path, the intermediate artifact produced, the validation gate applied, the output generated, and how that output is evaluated and mapped to the research questions.

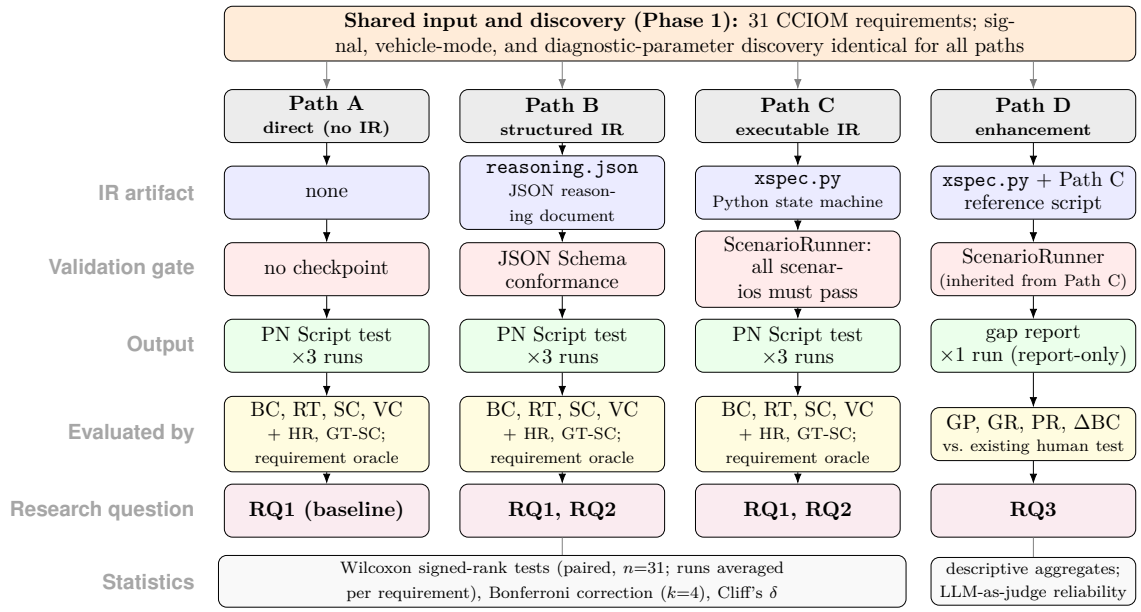


Figure 4.1: Side-by-side overview of the research activities

4.1 Research Approach

4.1.1 Design Science Research Methodology

This research follows the **six-step Design Science Research Methodology (DSRM)** of Peffers et al. [2]. DSRM is chosen because the work addresses a practical engineering problem — generating correct and complete test scripts from natural-language requirements — by constructing and empirically evaluating a technical *artifact*, rather than solely observing naturally occurring phenomena. Among the four DSRM entry points defined by Peffers et al., this work uses the **problem-centered entry point**: the research begins with an industrial problem, proceeds through artifact design and development, and concludes with evaluation against research objectives.

The artifact produced is a *method* in DSRM terms: a purposive process for LLM-based test generation parameterized by intermediate representation type. Peffers et al. require that a DSRM artifact be rigorously described, demonstrated on at least one instance of the problem, and evaluated against the stated objectives. These requirements are met by Sections 4.3–4.5 (description), the application to 31 industrial requirements (demonstration), and Chapter 5 (evaluation).

The process is treated as a *single design–evaluate cycle*: Paths A–C are designed as parallel generation alternatives, while Path D is an enhancement-analysis tier that operates on generated references and existing human-authored tests. Paths A–C are evaluated on a common dataset so that observed differences are attributable to IR strategy. Path D is evaluated separately for augmentation utility. Table 4.1 maps each DSRM phase to its location in this thesis and to the research question(s) it serves.

Table 4.1: Mapping of DSRM phases [2] to thesis content.

DSRM Phase	Thesis Location
1. Problem identification & motivation	Chapter 1, Sections 1.1–1.2 (background, problem statement)
2. Define solution objectives	Chapter 1, Sections 1.2–1.3 (problem statement, research questions)
3. Design & development	Chapter 4, Sections 4.3–4.5 (framework, IR types, enhancement mode)
4. Demonstration	Chapter 4, Section 4.3 (applied to 31 CCIOM requirements)
5. Evaluation	Chapter 4, Section 4.6; Chapter 5 (results); Chapter 6 (interpretation and threats to validity)
6. Communication	Chapter 6 (relation to prior work, industrial implications); Chapter 7 (conclusion and future work)

4.1.2 Experimental Design

The experimental component adopts a **within-subjects controlled comparison design**. All factors are held constant except the independent variable of interest — the type and formality of intermediate representation — and the effect on test generation quality is measured. This design eliminates confounds that arise when different systems use different prompting strategies, retrieval mechanisms, or output formats. The four experimental paths are:

- **Path A: Direct-to-TestScript (No-IR)** (`ir=none`, `mode=generate`): Direct generation from the requirement text and metadata, with no intermediate artifact.
- **Path B: Structured Plan IR (JSON)** (`ir=JSON`, `mode=generate`): Generation via a structured JSON reasoning document (`reasoning.json`) validated against a JSON schema.
- **Path C: Executable Behavioral IR (xSpec)** (`ir=Python`, `mode=generate`): Generation via an executable Python state machine specification (xSpec). All declared scenarios must pass before proceeding.

  The fourth path operates in a different mode and is evaluated separately from the IR comparison (RQ1/RQ2):

- **Path D: Enhancement** (`ir=Python`, `mode=enhance`): Gap-directed augmentation of an existing human-authored test using the executable xSpec as the behavioral reference. xSpec was chosen because augmentation requires a behavioral reference whose internal consistency has been validated before it is used to judge an existing test. Path D produces a structured report; it does not generate a test script from scratch and is not compared against Paths A–C.

4.1.3 Quality Dimensions and RQ Mapping

We measure four primary quality dimensions across Paths A–C (metrics are formally defined in Section 4.6.3):

- **Behavioral Completeness (BC):** the fraction of requirement-specified behaviors reproduced in the generated test, evaluated against the requirement text.
- **Requirement Traceability (RT):** the proportion of generated test behaviors that can be traced back to a passage in the requirement, penalizing fabricated content.
- **Signal Correctness (SC):** correctness of signal names, directions, frames, and nodes in the generated output.
- **Value Correctness (VC):** correctness of signal enumeration values and diagnostic parameter values used in the generated output.

We additionally report two non-primary metrics:

- **Hallucination Rate (HR):** the fraction of generated behaviors that the evaluator classifies as untraceable; reported as a descriptive complement to RT and not subjected to inferential testing.
- **Ground-Truth Scenario Coverage (GT-SC):** a secondary triangulation metric that compares generated tests against the human-authored test scripts; discussed in Section 4.6.2.

For the enhancement path (Path D), four additional metrics are collected: **Gap Precision (GP)**, **Gap Recall (GR)**, **Preservation Rate (PR)**, and **Coverage Delta (ΔBC)**.

The mapping to research questions is as follows:

- **RQ1 (IR vs. none):** Compare the pooled results of Paths B and C against Path A on BC, RT, SC, and VC.
- **RQ2 (Structured vs. Executable IR):** Compare Path C against Path B on the same metrics.
- **RQ3 (Augmentation):** Evaluate Path D in isolation on GP, GR, PR, and ΔBC.

4.2 Dataset

4.2.1 Source and Context

The evaluation dataset is drawn from Volvo Group's systems engineering tool, which manages the vehicle electrical/electronic (E/E) architecture hierarchically: from top-level vehicle variants through functional domains, Logical Design Components (LDCs; the organization's unit of functional decomposition, grouping the requirements of one related function area such as body level control, cf. Section 2.2), and individual requirement items. Each functional requirement (item type RFRQ) specifies the expected behavior of a software function in terms of input signals, output signals, preconditions, and expected outputs. Test cases are maintained in a proprietary DSL format designed for executable HIL and SIL test sequences.

The candidate pool was collected by running a scope collector script against the SE-Tool API for two Requirements Architecture Trees (RATs). A RAT is the tree of requirement and test items maintained for one electronic control unit (ECU) in the systems engineering tool. The two RATs used here belong to two body/cabin ECUs, referred to by their internal designations CCIOM and SCIM, both situated within the T3 (body/cabin electronics) architecture tier. The script traverses the test specification tree, resolves requirement-to-test-case traceability links, fetches requirement descriptions and test case PN Scripts, and records LDC assignments. This automated collection yielded 72 unique requirements (33 CCIOM, 39 SCIM) linked to 185 unique test cases through 226 requirement–test-case pairs; this constitutes the complete population of functional requirements reachable through the two RATs' test specification trees before any filtering.

4.2.2 Selection Criteria

The following criteria governed the filtering from the 72-requirement candidate pool to the final evaluation set.

Inclusion Criteria

- The item is a functional requirement (type RFRQ).
- At least one existing test case with a verified traceability link to the requirement can be resolved from the SE-Tool metadata.
- The requirement specification contains at least one extractable condition–response statement (not merely a title stub or placeholder).

Exclusion Criteria

- Meta-requirements or process-level requirements that lack testable input-output behavior.

- Requirements whose associated test cases could not be retrieved due to access restrictions or broken references in the SE-Tool.
- Duplicate requirements identified by identical specification content with differing handles.

The filtering pipeline is summarized in Table 4.2.

Table 4.2: Dataset filtering pipeline

Stage	Dropped	Remaining
Candidate pool (CCIOM + SCIM RATs)	–	72
No verified requirement–test trace link	17	55
All linked SCIM scripts are stubs[a]	23	32
One linked CCIOM script is a stub	1	31

[a] A *stub* is a placeholder test script (here under 700 B) with no behavior blocks; it encodes no human-authored coverage and cannot anchor the GT-SC or Path D comparisons, so its requirement is excluded.

The final dataset comprises **31 functional requirements** from a single LDC within the CCIOM RAT. All 31 requirements operate within the same signal ecosystem (shared CAN/LIN buses, common ECU boundary), eliminating inter-domain variation as a confounding factor while providing diversity in specification complexity and behavioral characteristics.

4.2.3 Dataset Characterization

As all 31 requirements reside within a single LDC, the number of discovered external CAN/LIN signals is nearly constant across requirements. Signal count therefore cannot discriminate complexity within this dataset. Complexity classification instead uses the specification length (l), defined as the character count of the requirement description text extracted from the SE-Tool:

- *Simple* ($l \leq 200$): short specifications describing a single signal-response relationship or a direct value mapping.
- *Medium* ($200 < l \leq 1{,}000$): specifications with multiple conditions, mode dependencies, or multi-step logic.
- *Complex* ($l > 1{,}000$): detailed specifications involving multiple interacting conditions, timing constraints, and extensive mode-dependent behavior.

The three tiers are roughly balanced, with simple and medium each comprising approximately a third of the dataset. Simple requirements typically specify single-threshold comparisons or direct value mappings. Complex requirements involve multi-condition logic (e.g., charger connection state machines, sensor freeze detection with multiple fallback strategies) and tend to produce the longest ground-truth test scripts.

Table 4.3: Dataset complexity distribution.

Complexity	Count	%	Spec Length	GT Script Size
Simple ($l \leq 200$)	11	35.5%	0–192 chars	1,901–26,852 bytes
Medium ($200 < l \leq 1{,}000$)	11	35.5%	223–835 chars	5,213–27,583 bytes
Complex ($l > 1{,}000$)	9	29.0%	1,118–3,252 chars	6,705–46,518 bytes
Total	31	100%		

4.3 The Unified Generation Framework

We implement all paths in a single parameterized generation skill rather than separate tools (Figure 4.2). The framework takes six inputs: a requirement handle, the IR type (`none`, `JSON`, `Python`), the generation mode (`generate` or `enhance`), an optional existing test handle, the vehicle architecture tier, and an output directory. The workflow then follows four phases:

1. **Phase 1 — Information Gathering (shared).** Every generation run begins with an identical discovery phase in which specialized agents extract contextual information from the systems engineering tool. This phase is parallelized in three waves to balance throughput and data dependencies:

   - *Wave 1:* requirement extraction, vehicle mode identification, and diagnostic object discovery execute concurrently. These agents have no mutual dependencies.

   - *Wave 2:* signal discovery traces input and output signals to their true external sources at the ECU boundary, a process that requires the architectural context established in Wave 1.

   - *Wave 3:* signal state enumeration and service qualifier resolution run concurrently, both depending on the signal list produced in Wave 2.

   The discovery agents are summarized in Table 4.4. Each agent is assigned a model tier commensurate with its task complexity: the signal discovery agent, which must perform recursive upstream tracing through routing components, uses the most capable model tier; requirement and diagnostic discovery use a mid-tier model; and lightweight enumeration tasks (signal states, vehicle modes, service qualifiers) use the most efficient model tier.

2. **Phase 2 — IR Generation (path-specific).** After discovery, the workflow diverges according to the IR type:

   - **Path A (No-IR):** Skip IR generation; pass discovery outputs directly to the test script generator.

   - **Path B (Structured IR):** Produce a JSON reasoning document (rea-

Table 4.4: Discovery agents and responsibilities.

Agent	Responsibility	Tier
discovery-requirement	Extract requirement content, architecture, ECU/LDC hierarchy	Mid
discovery-signal	Trace signals to external CAN/LIN sources at ECU boundary	High
discovery-signal-states	Enumerate default values and failure modes for external signals	Efficient
discovery-vehicle-modes	Identify applicable vehicle modes	Efficient
discovery-doid	Resolve diagnostic object IDs (DOIDs)	Mid
discovery-service-qualifier	Map DOIDs to service qualifiers (HIL only)	Efficient

soning.json) organizing signals, parameters, behaviors, and coverage. Validate it against a JSON schema.

- **Path C (Executable IR):** Produce a Python xSpec module (`xspec.py`) encoding states, transitions, and behaviors. Execute the ScenarioRunner on all declared scenarios; only if all pass is the xSpec considered valid. This validation checkpoint acts as a semantic gate that has no analogue in Paths A or B.

- **Path D (Enhance):** A full Path C pipeline produces a reference test script. The existing human-authored test and the generated reference are both parsed through the same ANTLR-based parser, producing two coverage descriptions at an identical structural abstraction level. A symmetric comparison agent identifies matches, gaps, and value errors between them (Section 4.5).

3. **Phase 3 — Test Script Generation.** A single generator produces the final test script in the DSL. It takes the selected IR (or none) and the discovery outputs as input. Using one generator across all paths ensures that output differences arise solely from the IR strategy.

4. **Phase 4 — Output Artifacts.** Write the final test script and any intermediate artifact (the JSON or xSpec) to the output directory, along with validation logs. Retaining these artifacts supports evaluation and reproducibility.

4.3.1 PN Script DSL Format

All four paths generate output in the PN Script domain-specific language, the proprietary test scripting format used by the organization for executable HIL and SIL test sequences (see Chapter 2, Listing 2.1 for a representative excerpt). A PN Script test case consists of a preparation sequence (variable declarations), a main sequence of `StateChange` blocks, each containing `Set`, `Check`, and `Wait` commands, grouped

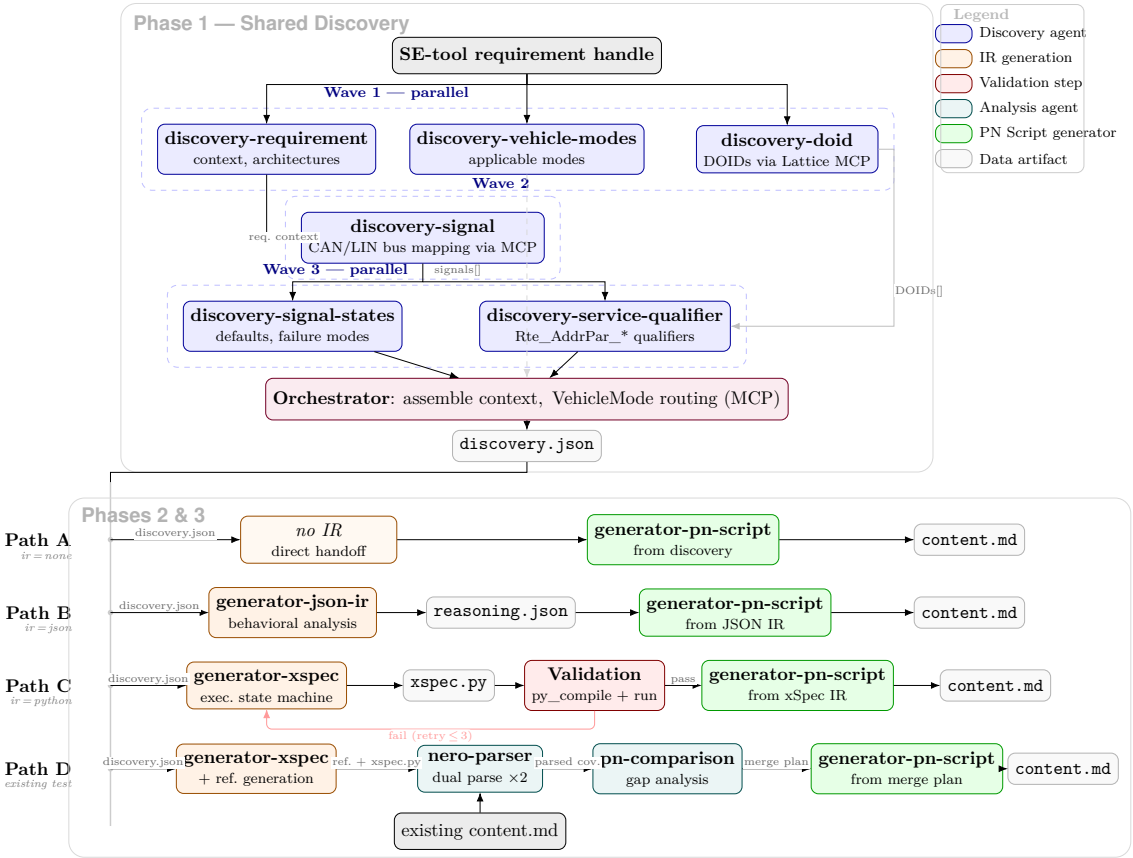


Figure 4.2: Unified four-path pipeline

by `Comment` case markers.

4.4 Intermediate Representation Levels

This section describes the three IR types used: No-IR (Path A), Structured Plan IR (Path B), and Executable Behavioral IR (Path C). A comparative summary is given in Table 4.5.

Given the generalizability constraints identified in Section 2.4, we select JSON and Python as the IR languages for this work. JSON is native to LLM training corpora, making generation reliable, and schema validation provides a structural checkpoint. Python is the most widely represented programming language in LLM training data, avoiding the "syntax cliff" that afflicts UML or Simulink notations, while the `transitions` library [38] provides enough state-machine expressiveness to encode behaviors, guards, and scenarios that can be executed for self-verification. This trades formality—a Python class is less rigorous than a mathematically defined automaton—for pragmatism: the LLM can generate valid instances with high reliability, and the resulting artifacts can be validated automatically.

The distinction between the two IR types rests on whether they can verify their own internal consistency. A *structured intermediate representation* captures requirement semantics in structured form (JSON with schema validation) but has no mechanism to check whether the captured behaviors are logically consistent, complete, or feasible. An *executable intermediate representation* encodes semantics in a format that can be executed and validated—scenarios declared in the specification are run against the state machine, and scenarios that fail indicate specification errors requiring revision.

4.4.1 Direct-to-TestScript (No-IR) — Path A

In Path A, the LLM receives only the requirement text and discovered metadata (signals, modes, DOIDs) and produces the test script in one step. No intermediate artifact is created, and no validation checkpoint exists between discovery and generation. This path serves as the baseline for RQ1.

4.4.2 Structured Plan IR (Path B)

The Structured Plan IR is a structured JSON document (`reasoning.json`) that decomposes the requirement into a machine-readable format. Its contents include:

- Requirement metadata: handle, name, architecture tier, ECU context.
- Signal analysis: listings of input/output signals with node, bus, frame, value range, and defaults.
- Diagnostic parameters (DOIDs): value types and defaults.

- Behaviors: an array of objects, each with an *id*, natural language description, category, trigger type, condition, preconditions, expected output, applicable modes, and rationale.
- Vehicle mode analysis: applicable vs non-applicable modes.
- Coverage plan: count of behaviors, distribution by category, coverage notes.

Listing 4.1 shows a representative behavior entry from a `reasoning.json` artifact.

```
{
  "id": "b1",
  "description": "Both sensors valid, Temp1 > Temp2:
     BatteryTemp = MAX(T1, T2)",
  "category": "nominal_behavior",
  "trigger_type": "state",
  "trigger_condition":
      "LIN_BatteryTemp1_1 valid AND LIN_BatteryTemp2_1 valid
         AND Temp1 > Temp2",
  "expected_output":
      "BatteryTemp = LIN_BatteryTemp1_1 (raw 75 => 30 degC)",
  "preconditions": ["applicable vehicle mode set"],
  "rationale":
      "Direct application of MAX selection logic: requirement
         clause
       'BatteryTemp = MAX(LIN_BatteryTemp1, LIN_BatteryTemp2)
          '"
}
```

Listing 4.1: `reasoning.json` excerpt: one behavior entry for Battery Temperature Selection. The entry records the trigger condition, expected output, and traceability rationale.

4.4.3 Executable Behavioral IR (xSpec) — Path C

The Executable Behavioral IR is a Python module (`xspec.py`) encoding the requirement semantics as a finite state machine. It is executable and its internal consistency is validated before downstream use. The design draws on model-based testing principles [22] while using Python as the specification language to avoid the "syntax cliff" [35, 24]. Its components include:

- *BaseXSpec*: an abstract base class that provides the state machine infrastructure. Each concrete specification declares its states as a list of strings and its initial state. The constructor instantiates a finite state machine using the transitions library [38], a lightweight Python library for declarative state machine definitions. The base class also provides edge-tracking utilities: methods to initialize tracking for a named signal, detect value changes (rising and falling edges), and update tracked values. This edge-detection capability — absent in the structured IR—enables the specification to distinguish between "signal X has value V" (level) and "signal X changed to value V" (edge), a distinction that is critical for automotive test correctness.

- *Behavior catalogue*: a dataclass that represents a single testable behavior with fields for identifier, natural language description, category (drawn from the same seven-category vocabulary as the structured IR), trigger condition, expected output, preconditions, and applicable vehicle modes. The use of the same category vocabulary across both IR types enables direct comparison of behavioral coverage.

- *ScenarioRunner*: A test harness that executes declared scenarios (annotated with `@runner.scenario`) following a Setup–Stimulus–Verify pattern. The scenario sets initial signal values and state, applies a stimulus, calls the specification's update method to advance the state machine, and then asserts expected outputs and state transitions. The runner executes all declared scenarios and reports pass/fail results.

- *VehicleMode model*: Enumeration of the vehicle's power/operating modes, consistent with domain conventions. These modes describe the electrical state of the vehicle and gate which functions are active: *Parked* (vehicle shut down and locked, only always-on functions powered), *Living* (occupants present without ignition; cab comfort functions powered), *Accessory* (ignition in the accessory position, infotainment and convenience functions powered but the engine off), *Pre-Running* (power-up sequence preparing the vehicle to start), *Cranking* (engine start in progress, starter engaged), and *Running* (engine running and the vehicle fully operational). Many requirements specify behavior only for a subset of these modes.

Listing 4.2 shows a representative excerpt of an xSpec class, illustrating the state declarations and one validated scenario.

```
class BatteryChargingVoltageRegulationSpec:
    states = [
        'normal',           # Temperature valid; computing from
            current value
        'using_last_valid',# Temperature Error/NA; within < 10
            min window
        'using_fallback',  # Temperature Error/NA; >= 10 min
           window
    ]
    initial_state = 'normal'

    def _register_behaviors(self):
        self.behaviors = [
            Behavior('b1', 'Temp␣at␣lower␣plateau␣(-40C)␣->␣
               29.2V',
                     category='boundary_values',
                     trigger_condition='AmbientAirTemperature␣
                        =␣-40C',
                     expected_output='WantedLBChargingVoltage␣
                        =␣29.2V'),
            Behavior('b9', 'Error/NA␣<␣10␣min␣->␣use␣last␣
               valid␣value',
                     category='fault_conditions', ...),
        ]
```

```
@runner.scenario('b1: Lower plateau -- AmbientAirTemperature
    at -40 degC')
def test_lower_plateau(spec):
    spec.AmbientAirTemperature = 7456  # raw for -40 degC
    spec.update()
    assert spec.WantedLBChargingVoltage == 2920  # raw for
        29.2 V
```

Listing 4.2: xSpec excerpt: Battery charging voltage regulation without battery sensor. The states model the three operating regimes; the scenario validates the nominal temperature-to-voltage mapping.

Table 4.5: Comparison of IR properties (Paths A–C).

Property	No IR (A)	Structured IR (B)	Executable IR (C)
Artifact type	None	JSON reasoning document	Python xSpec module
Machine-readable	N/A	Yes	Yes
Schema-validated	N/A	Yes (JSON Schema)	Yes (Python types)
Executable/Validatable	N/A	No	Yes (scenario execution)
Validation method	None	Schema conformance	Scenario execution
State-machine semantics	(None)	Not represented	Explicit (transitions)
Edge detection	(None)	None	Verified
Timing logic	(Implicit in output)	Described (text)	Executable guards
Reproducibility	Low	Medium	High

4.5 Enhancement Mode

The enhancement path addresses RQ3 by investigating whether IR-guided gap analysis can systematically improve existing human-authored test scripts without discarding correct content. Unlike Paths A through C, which generate tests from scratch, Path D takes an existing test script as additional input and produces a structured enhancement report. In this thesis, Path D is evaluated in report-only mode; no merged script is automatically emitted.

A key design decision is the use of **symmetric dual-parse comparison**. Earlier iterations of this pipeline compared parser output (structural behaviors from the existing test) against xSpec output (logical behaviors from the executable specification) — two representations at different abstraction levels. This cross-abstraction comparison made value-error detection difficult because the granularity and representation format differed between the two sides. The current design instead generates a reference test script through the full Path C pipeline, then parses both scripts through the same ANTLR-based parser, producing two coverage descriptions at an identical structural abstraction level. This enables field-by-field comparison and, critically, value-error detection.

The enhancement pipeline consists of five stages:

1. **Obtain reference script.** A reference test script is generated from the re-

quirement using the full Path C pipeline: xSpec generation with mandatory validation, followed by test script generation.

2. **Fetch existing test.** The existing human-authored test script is retrieved from the systems engineering tool.

3. **Dual parse.** Both the existing test (Stage 2) and the generated reference (Stage 1) are parsed through the same ANTLR-based parser. The parser extracts, for each script independently: (a) the behaviors tested (individual StateChange-and-check blocks); (b) the signals manipulated and checked with their specific values; (c) the vehicle modes exercised; (d) the diagnostic parameters used; (e) the initialization patterns (wake-up sequences, mode transitions, wait durations); and (f) a structured coverage summary. The two parse operations run in parallel and produce coverage objects with identical schemas.

4. **Symmetric comparison.** A dedicated comparison agent receives both parsed coverage objects and the requirement context. The agent performs three analyses:

   (a) Signal fingerprinting: matching behaviors across the two scripts based on the signals they manipulate and assert.

   (b) Field-by-field value comparison on matched behaviors: disagreements are flagged as *value errors* — cases where the existing test checks for a different output value than the generated reference.

   (c) Gap and legacy identification: behaviors in the generated reference but absent from the existing test are classified as gaps; behaviors in the existing test but absent from the generated reference are classified as existing-only.

   The comparison output includes matched behaviors (with value-match status), gaps (with priority and rationale), existing-only behaviors, value errors, and a recommendation class (for example, extend or review).

5. **Report generation.** The orchestration writes a structured analyst report for each requirement containing detected gaps, preservation/loss indicators, and coverage comparison outputs used by GP, GR, PR, and ΔBC evaluation.

4.6 Evaluation Design

4.6.1 Experimental Procedure

Each of the 31 requirements in the dataset (Section 4.2) is processed through all applicable generation paths. Paths A, B, and C are applied to every requirement.

To account for the inherent nondeterminism of LLM generation, each requirement-path combination is executed **three** times, producing three independent outputs per configuration. Metrics are computed for each run individually and then aggregated (mean and standard deviation) to characterize both central tendency and variability. This repeated-measures design enables the use of the Wilcoxon signed-rank test [39] for statistical comparison between paths.

The following variables are held constant across all paths and runs:

- **LLM family:** all agents use models from the Claude family (Anthropic), assigned to three capability tiers commensurate with task complexity. The signal discovery agent uses Claude Opus 4 (`claude-opus-4-6`); all generator and analysis agents use Claude Sonnet 4 (`claude-sonnet-4-6`); lightweight discovery agents use Claude Haiku 4.5 (`claude-haiku-4-5-20251001`). The orchestrator uses Sonnet. All models run at the Anthropic API default temperature ($t = 1.0$) with no top-p override. A lower temperature would reduce output variance but was not overridden, as the industrial platform uses API defaults and the repeated-measures design (three runs per configuration) is specifically intended to characterize and account for generation nondeterminism rather than suppress it.
- **Discovery data:** Phase 1 is executed once per requirement, and the resulting discovery artifacts are reused across all paths and runs.
- **Output format:** all paths produce test scripts in the same PN Script DSL.
- **Prompt structure:** the test script generator agent uses the same prompt template across all modes, varying only the input artifacts.

The total number of generation runs is: 31 requirements $\times$ 3 paths (A, B, C) $\times$ 3 runs = 279 runs for RQ1 and RQ2, plus additional runs for Path D (RQ3) on the subset with existing tests.

4.6.2 Ground Truth and Comparison Basis

The evaluation employs a **dual-oracle** design that distinguishes between two complementary reference points, reflecting different definitions of test quality.

Primary Oracle: the Requirement The primary oracle for RQ1 and RQ2 is the **requirement text itself**, as resolved from the SE-Tool and stored in the discovery artifact (`discovery.json`). A requirement-centric evaluator agent (described in Section 4.6.3) reads the requirement body alongside the generated test script and measures quality directly against the specification: whether the generated test covers the behaviors the requirement mandates, whether the signals and values used are correct, and whether any generated content is not traceable to the requirement.

This design avoids a fundamental validity problem with using human-authored tests as the sole oracle: a generated test that correctly covers a behavior the human

engineer *missed* would be penalized under a human-script-comparison approach. The requirement is the authoritative specification; the human-authored test is one possible implementation of it.

Secondary Reference: Human-Authored Tests The human-authored test cases maintained in the SE-Tool serve as a **secondary reference** for triangulation. These test cases were written by domain engineers with knowledge of the vehicle platform, the signal architecture, and the diagnostic infrastructure. They represent current industrial practice and have been used in production HIL and SIL testing.

The secondary metric **GT-SC** (Ground-Truth Scenario Coverage, Section 4.6.3) measures how closely the generated tests align with what human engineers wrote. A GT-SC result that diverges from the requirement-centric BC result is itself a finding: it reveals the gap between the requirement as written and the test as currently implemented by engineers.

It should be noted that human-authored tests are not necessarily optimal: they may contain redundant steps, miss boundary conditions, or reflect outdated signal information. Where generated tests cover behaviors absent from the human-authored ground truth, this is recorded as potential novel coverage for qualitative analysis.

RQ3 Oracle: Enhancement Against the Existing Test For Path D (RQ3), the human-authored test script is the object under study and the pipeline augments it. The evaluation therefore uses the human-authored test as the reference for gap detection (GP, GR), preservation measurement (PR), and coverage improvement (ΔBC). The requirement remains the oracle for classifying whether an identified gap is genuine.

4.6.3 Metrics

The primary metrics for RQ1 and RQ2 are computed by a **requirement-centric evaluator**, an LLM-based agent that reads the generated test script and the **discovery.json** artifact for a given requirement. The agent identifies behaviors on both sides automatically: on the requirement side, it extracts expected behaviors by parsing conditional clauses, boundary conditions, failure modes, and state transitions from the requirement text; on the generated-script side, it identifies tested behaviors as individual `StateChange`-and-`Check` blocks demarcated by `Comment` case markers in the PN Script (Section 4.3.1). The agent then produces a structured **evaluation.json** containing per-metric scores, per-behavior traceability classifications, signal error records, and value error records. Using a dedicated evaluator agent — rather than comparing against the human-authored test — anchors the evaluation to the authoritative specification and avoids the oracle validity problem described in Section 4.6.2. Because this evaluator is an LLM-as-a-judge instrument rather than a human rater, its reliability is itself a direct threat to construct and conclusion validity; we therefore assess its design in Section 4.6.4 and empirically validate its measurements against independent human raters in the manual validation study reported in Section 5.2.

Primary Metrics (Paths A–C) The following five metrics are applied to all three generation paths and used for the RQ1 and RQ2 comparisons.

Behavioral Completeness (BC). The fraction of requirement-specified expected behaviors that are covered in the generated test script. The evaluator agent extracts the set of expected behaviors E_{req} from the requirement text by identifying conditional clauses, boundary conditions, failure modes, and state transitions. Each expected behavior is then classified as *covered* (a generated block directly tests it), *partially covered* (the right signals but wrong values, or vice versa), or *missing*.

Let $|E_{\text{req}}|$ denote the total number of expected behaviors, |TP| the number covered, and |part| the number partially covered:

$$\text{BC} = \frac{|\text{TP}| + 0.5\,|\text{part}|}{|E_{\text{req}}|}$$

where TP (true positive) is a covered expected behavior, FN (false negative) is a missing expected behavior, and partial coverage receives half credit. The 0.5 weight is a pragmatic midpoint: partial coverage reflects useful but incomplete alignment (for example, correct signal with incorrect value), and is therefore penalized relative to full coverage without being treated as a total miss. BC is a recall-oriented metric: it measures whether the generated test *covers* what the requirement mandates, not whether the generated test contains anything extra.

Requirement Traceability (RT). The fraction of generated test behaviors that can be attributed to the requirement. The evaluator classifies each generated behavior $b \in B_{\text{gen}}$ as:

- *Traceable (T):* a specific passage in the requirement justifies this behavior.
- *Inferred (I):* the behavior is a reasonable test engineering practice (e.g., boundary test, fault injection) implied by the requirement structure but not explicitly stated.
- *Untraceable (U):* no requirement passage or structural inference justifies this behavior (hallucination).

$$\text{RT} = \frac{|T| + 0.5\,|I|}{|B_{\text{gen}}|}$$

RT is a precision-oriented metric: it penalizes hallucinated content. Untraceable behaviors (U) correspond to false positives — generated content without any requirement basis. Because inferred and untraceable can be difficult to separate in borderline cases, RT and HR are interpreted together; their absolute values should be treated as instrument-conditioned (Section 4.6.4).

Signal Correctness (SC). The proportion of signal references in the generated test script that are correct with respect to the discovered signal list:

$$\text{SC} = \frac{|S_{\text{correct}}|}{|S_{\text{total}}|}$$

where S_{correct} includes signals with correct name, correct direction (input vs. output), correct frame, and correct node. S_{total} is all signal references in the generated test. An incorrect signal reference is a false positive (FP): the generated test uses a signal name, direction, or frame that does not match the discovery data. SC detects hallucinated signals — a known failure mode in LLM-generated domain-specific code.

Value Correctness (VC). The proportion of value references in the generated test script that are correct with respect to the signal enumeration in the discovery artifact:

$$\text{VC} = \frac{|V_{\text{correct}}|}{|V_{\text{total}}|}$$

where V_{correct} includes values that exist in the signal's enumerated value set from discovery (by name or numeric value) and DOID values whose hash and type match the discovery data. An incorrect value reference is a FP: the generated test asserts a value that does not correspond to any valid enumeration entry.

Hallucination Rate (HR). The fraction of generated test behaviors that the requirement-traceability pass classifies as untraceable:

$$\text{HR} = \frac{|U|}{|B_{\text{gen}}|}$$

where U is the set of behaviors for which no requirement passage and no structural inference justifies the behavior. HR is reported as a descriptive complement to RT and is not subjected to inferential testing; a high HR indicates that the generator fabricates content that cannot be linked to the specification.

Ground-Truth Scenario Coverage (GT-SC). The proportion of human-authored ground-truth behaviors that are reproduced in the generated test script:

$$\text{GT-SC} = \frac{|B_{\text{gen}} \cap B_{\text{GT}}|}{|B_{\text{GT}}|}$$

where B_{GT} is the set of behaviors extracted from the human-authored test and B_{gen} is the set from the generated test. Behavior matching uses signal-fingerprint-based similarity (as described in Section 4.5, Stage 4) with a match threshold of 80%.

GT-SC is reported as a *secondary* metric. A low GT-SC does not necessarily indicate a low-quality generated test: the human-authored test may itself be incomplete, and the generated test may correctly cover behaviors the human missed. GT-SC is most useful as a triangulation signal — when GT-SC and BC point in the same direction, confidence in the finding increases; when they diverge, the divergence itself is a finding about the gap between current industrial test practice and the requirements as written.

Table 4.6: Metrics summary per research question.

Metric	Type	RQ1	RQ2	RQ3
Behavioral Completeness (BC)	Primary	✓	✓	
Requirement Traceability (RT)	Primary	✓	✓	
Signal Correctness (SC)	Primary	✓	✓	
Value Correctness (VC)	Primary	✓	✓	
Hallucination Rate (HR)	Descriptive	✓	✓	
GT-SC	Secondary	✓	✓	
Gap Precision (GP)	Primary			✓
Gap Recall (GR)	Primary			✓
Preservation Rate (PR)	Primary			✓
Coverage Delta (ΔBC)	Primary			✓

Enhancement-Specific Metrics (Path D) The following metrics are specific to the enhancement path (Path D):

Gap Precision (GP). The proportion of gaps identified by the analysis that correspond to genuine coverage omissions in the existing test:

$$\mathrm{GP} = \frac{|\mathrm{TP}_{\mathrm{gap}}|}{|\mathrm{TP}_{\mathrm{gap}}| + |\mathrm{FP}_{\mathrm{gap}}|}$$

where $\mathrm{TP}_{\mathrm{gap}}$ (true positive) is an identified gap confirmed as a genuine omission by an independent evaluator, and $\mathrm{FP}_{\mathrm{gap}}$ (false positive) is a flagged gap that the independent evaluator deems already adequately covered or not required by the specification.

Gap Recall (GR). The proportion of genuine coverage omissions that are successfully identified:

$$\mathrm{GR} = \frac{|\mathrm{TP}_{\mathrm{gap}}|}{|\mathrm{TP}_{\mathrm{gap}}| + |\mathrm{FN}_{\mathrm{gap}}|}$$

where $\mathrm{FN}_{\mathrm{gap}}$ (false negative) is a genuine omission that the analysis failed to identify. A gap is classified as genuine if an independent evaluator confirms that the requirement mandates the behavior and the existing test does not cover it. In this study, the independent evaluator is a separate LLM session (LLM-as-judge [40]) rather than a human rater, making the resulting GP and GR measures of LLM-vs-LLM agreement rather than human-validated accuracy. This design choice reflects the cost of manual gap classification across 96 candidates and 29 requirements; the resulting metrics should be interpreted as indicative agreement rates rather than ground-truth-validated scores. The requirement-anchored classification mitigates circular reasoning — the oracle is the requirement itself, not the generated reference that identified the gap.

Preservation Rate (PR). The proportion of existing correct behaviors in the human-authored test that are retained in the enhanced output:

$$\text{PR} = \frac{|B_{\text{preserved}}|}{|B_{\text{existing_correct}}|}$$

A preservation rate below 100% indicates that the enhancement process has inadvertently removed or degraded existing correct content. FN in this context is an existing correct behavior that was lost during enhancement.

Coverage Delta (ΔBC). The net change in requirement-level behavioral completeness achieved by the enhancement:

$$\Delta\text{BC} = \text{BC}_{\text{enhanced}} - \text{BC}_{\text{existing}}$$

where $\text{BC}_{\text{existing}}$ is the BC score of the original human-authored test computed against the requirement, and $\text{BC}_{\text{enhanced}}$ is the BC score of the augmented output computed identically. A positive ΔBC indicates that the enhancement added genuine requirement coverage; a negative value would indicate that the merge process inadvertently removed more behaviors than it added. This metric directly captures the net improvement in test quality as measured against the authoritative specification.

4.6.4 Metric Instrument Validation

Several of the metrics above depend on automated LLM-based judgements, creating a risk of circular evaluation. To bound this risk, we conduct a **manual validation study** in which two human raters independently score the same generated outputs that the automated evaluator scored, and we compare the resulting assessments.

A sample of **6 requirements** is drawn from the dataset to span the three complexity tiers, with the exact handle list jointly fixed by the two raters before either of them opens a packet. One Path C run per requirement is selected, yielding 6 generated output files. Each rater receives a validation packet containing the requirement text, the agent's extracted expected behaviors, and the generated script.

BC extraction check. For each of the 6 sampled requirements, each rater independently: (i) reads the requirement text and lists all expected behaviors they can identify; (ii) classifies each agent-extracted behavior as *correctly extracted* or *spurious*, and each human-extracted behavior as *captured* or *missed by agent*. From these classifications, the evaluator instrument's precision and recall are computed per handle:

$$\text{Eval Precision} = \frac{|\text{correctly extracted}|}{|\text{agent-extracted}|}, \qquad \text{Eval Recall} = \frac{|\text{captured}|}{|\text{human-extracted}|}$$

If the evaluator's precision or recall falls below 0.85, the BC metric is interpreted as instrument-limited and discussed with explicit caveats in the results and discussion chapters; otherwise the automated scores are treated as sufficiently reliable for

primary reporting. The remaining metrics (RT, SC, VC) are not independently validated against human raters in this study; their absolute values should be interpreted as instrument-conditioned, while relative path comparisons remain informative because any systematic bias is constant across paths. The 0.85 threshold is used here as a pragmatic reliability cutoff for this study and is not presented as a universal psychometric standard.

4.6.5 Statistical Analysis

For RQ1 and RQ2, pairwise comparisons between paths are conducted using the Wilcoxon signed-rank test, a non-parametric test appropriate for paired ordinal or continuous data that does not assume normality [39]. Effect sizes are reported using Cliff's delta [41], with $|\delta| < 0.147$ considered negligible, $|\delta| < 0.33$ small, $|\delta| < 0.474$ medium, and $|\delta| \geq 0.474$ large [42]. Significance is assessed at $\alpha = 0.05$ with Bonferroni correction for multiple comparisons.

For RQ1, two analyses are conducted. The primary pooled comparison tests Paths B and C (any IR) against Path A (no IR), with Bonferroni correction across the four primary metrics (BC, RT, SC, VC), i.e., $k = 4$. A co-primary disaggregated analysis then tests Path B versus Path A and Path C versus Path A separately to determine whether both IR types contribute to any observed pooled effect. For each disaggregated comparison family, Bonferroni correction is again applied across the same four metrics ($k = 4$ per family).

For RQ2, the comparison is between Path B (structured IR) and Path C (executable IR) on the same requirements, constituting a paired within-subjects design.

For RQ3, the primary analysis is descriptive: GP, GR, PR, and ΔBC are reported per requirement and in aggregate, accompanied by a qualitative analysis of the types of gaps identified and the types of existing behaviors preserved.

Complexity-stratified reporting. All metrics for RQ1 and RQ2 are reported stratified by requirement complexity tier (simple, medium, complex; Section 4.2.3). This stratification serves two purposes: (i) it reveals whether the IR benefit is concentrated in complex requirements or uniformly distributed, and (ii) it guards against Simpson's paradox, where an aggregate effect could mask opposing trends within strata. Due to the small number of complex requirements ($n = 9$), the complex stratum is treated as exploratory and interpreted descriptively rather than subjected to inferential tests.

4.7 Ethical Considerations

All experimental data originates from Volvo Group's proprietary systems engineering environment and is handled under a non-disclosure agreement. No proprietary signal names, source code, or internal identifiers are disclosed in this thesis; all examples are anonymized or presented in generic form.

No personal data is collected or processed at any stage of this work. The dataset consists exclusively of functional requirements and test scripts authored by engineers; no information about individual authors is recorded or analyzed.

The LLM is the tool under study, not a human participant. The research evaluates the quality of machine-generated artifacts against existing engineering artifacts and does not involve human subjects experimentation. Ethics board approval is therefore not required under the applicable institutional guidelines.

All generated test scripts are evaluated offline through static analysis metrics. No generated artifact is deployed to safety-critical vehicle systems without prior human review and validation through the organization's standard release process.

5

Results

This chapter reports the outcome of the experimental evaluation defined in Section 4.6. Section 5.1 summarizes the experimental run and dataset coverage. Section 5.2 reports the manual validation study that establishes the reliability of the evaluation instruments. Sections 5.3, 5.4, and 5.5 answer RQ1, RQ2, and RQ3 in turn. Interpretation and implications are reserved for Chapter 6.

5.1 Experimental Run Overview

The full experimental procedure of Section 4.6.1 was executed on the 31-requirement dataset described in Section 4.2. Each requirement was processed through Paths A, B, and C three times, yielding 279 generated PN Script test cases. The pipeline produced output for all 93 (requirement × path) configurations in every run, with no permanent generation failures (Table 5.1).

Table 5.1: Generation completion across the three independent runs.

Run	Path A	Path B	Path C	Path D†	Total (A–C)
Run 1	31/31	31/31	31/31	29/31	122/124
Run 2	31/31	31/31	31/31	—	93/93
Run 3	31/31	31/31	31/31	—	93/93
Total	93/93	93/93	93/93	29/31	308/310

All 279 generated PN Scripts parsed without error against the DSL grammar (Syntax Validity = 100% on every path and run). Because SV is constant across paths it cannot differentiate them and is therefore not carried into the RQ1/RQ2 comparison tables; it is reported here as a generator-completion certification.

Path D (report-only enhancement analysis) was applied to all 31 requirements; 30 produced a complete ΔBC pair and 29 produced a usable analyst report. The two excluded handles arose from a single SDK-side timeout during Phase 2 scoring of the human-authored test union and a single transient analyst read failure; both are summarized in Section 5.5.

The Path C orchestrator's retry counters (first-attempt vs retried xSpec validation) were not persisted to `progress.csv` in this version of the pipeline; only the final per-run status was logged. All 93 Path C runs eventually produced a parseable PN Script and no xSpec required manual repair. A finer-grained accounting of first-attempt success vs retry rates is identified as instrumentation work in Chapter 7.

5.2 Manual Validation Study

The validation protocol of Section 4.6.4 was executed by the two thesis authors on a sample of 6 requirements (Appendix A.1). The scoring rubric, packet structure, a worked example, and the evaluation tooling are documented in Appendix A.2.

Table 5.2 compares the agent's BC extraction against the two human raters' independent assessments on the same 6 generated outputs.

Table 5.2: Human vs. agent BC extraction on 6 validated outputs (one Path C run per requirement).

Handle	**Tier**	n_{agent}	Confirmed
`2CE1FCDE`	simple	1	1/1
`28AA60CA`	medium	6	6/6
`28B31956`	medium	6	3/6
`28C3F3F0`	medium	8	8/8
`28AAD449`	complex	9	9/9
`28C3D7E0`	complex	9	9/9
Aggregate		39	36/39

5.2.1 BC Extraction

The evaluator agent extracted 39 behaviors across the 6 requirements. Both raters confirmed 36 as correctly extracted (Eval Precision = 0.923); the 3 spurious entries all belong to one handle (`28B31956`). Rater A independently identified 38 expected behaviors, of which 30 were captured by the agent (Eval Recall = 0.789); Rater B identified 35, of which 28 were captured (Eval Recall = 0.800).

The concentration of all three spurious extractions in `28B31956` has a clear explanation. This requirement specifies a battery-voltage low-pass (IIR) filter—a continuous, parametric signal-processing behavior rather than a set of discrete conditional behaviors. On this requirement the evaluator over-decomposed the filter specification, emitting separate "behaviors" for individual filter-coefficient configurations and frequency points that both raters judged to be facets of a single filtering behavior rather than independent requirement behaviors. The over-extraction is therefore a granularity disagreement specific to parametric signal-processing requirements, not a content hallucination: the spurious entries describe real aspects of the requirement but at a finer granularity than the raters counted. The remaining five requirements, whose behaviors are discrete and conditional, showed perfect extraction precision

(Table 5.2), indicating that the effect is localized to this requirement class rather than systematic.

5.2.2 Reliability Summary

The per-instrument outcomes against the 0.85 threshold of Section 4.6.4 are summarized in Table 5.3. BC Eval Precision passes for both raters; BC Eval Recall is borderline below. As stated in Section 4.6.4, the remaining metrics (RT, SC, VC) are not independently validated against human raters; their absolute values should be treated as instrument-conditioned, while relative path comparisons remain informative.

Table 5.3: BC extraction reliability against the 0.85 threshold of Section 4.6.4 (two-rater study, $n = 6$ requirements).

Instrument	Reliability statistic	Meets 0.85?
BC Eval Precision	$P_a = P_b = 0.923$	Yes
BC Eval Recall	$R_a = 0.789$, $R_b = 0.800$	Borderline

5.3 RQ1: Effect of Intermediate Representation

RQ1 asks whether using any intermediate representation (Paths B and C, pooled) improves test generation quality compared to direct generation (Path A). A co-primary disaggregated analysis tests Path B and Path C against Path A separately to determine whether both IR types contribute to any pooled effect. The primary requirement-centric metrics are reported first, followed by the GT-SC secondary triangulation against human-authored tests.

5.3.1 Primary Metrics: Requirement-Centric Evaluation

Table 5.4: Primary metrics across Paths A, B, C ($n = 31$, means ± SD; per-handle values are first averaged over the three runs).

Metric	Path A	Path B	Path C
BC (Behavioral Completeness)	0.798 ± 0.183	0.863 ± 0.152	0.906 ± 0.157
RT (Requirement Traceability)	0.803 ± 0.093	0.823 ± 0.096	0.895 ± 0.107
SC (Signal Correctness)	0.882 ± 0.051	0.986 ± 0.021	0.994 ± 0.027
VC (Value Correctness)	0.985 ± 0.010	0.990 ± 0.031	0.991 ± 0.022
HR (Hallucination Rate)	0.020 ± 0.010	0.005 ± 0.021	0.000 ± 0.000

5.3.2 Statistical Comparison (Pooled Analysis)

The pooled comparison treats Paths B and C as a single "IR" group and tests against Path A using the Wilcoxon signed-rank test (paired on requirement). Effect sizes are reported as Cliff's δ.

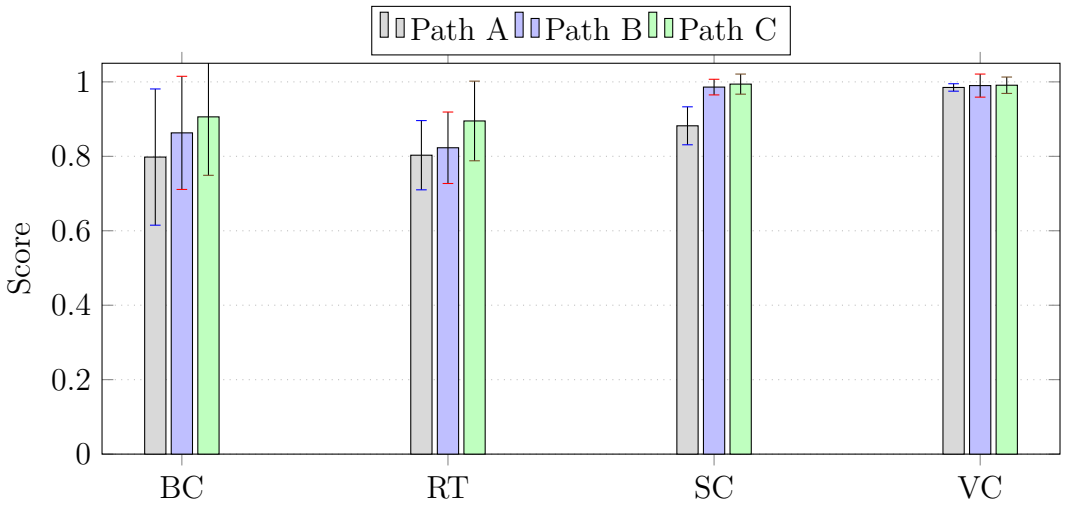


Figure 5.1: Primary-metric summary across paths (means with SD error bars, from Table 5.4).

Table 5.5: RQ1 pooled comparison: Paths B and C (averaged per requirement) versus Path A. Wilcoxon signed-rank test on per-requirement paired values; $p_{\text{adj}} = \min(1,\ p \cdot k)$ with $k = 4$ primary metrics. Cliff's δ uses the convention $\delta(\text{IR}, \text{A})$; positive values favor the IR paths.

Metric	Median diff. (IR−A)	W	p	p_{adj}	Cliff's δ
BC	+0.036	29.5	0.0006	0.0024*	+0.164
RT	+0.052	86.0	0.0010	0.0040*	+0.273
SC	+0.000	9.0	0.110	0.439	+0.053 (n.s.)
VC	+0.000	103.0	0.941	1.000	+0.211 (n.s.)

*Significant after Bonferroni correction at family-wise $\alpha = 0.05$.

BC and RT improve significantly after Bonferroni correction, while SC and VC do not. It is worth noting explicitly, however, that even for the significant metrics the effect sizes are *small*: the largest is RT at Cliff's $\delta = +0.273$, and BC is +0.164, both at or below the $|\delta| = 0.33$ small/medium boundary used in this thesis (Section 4.6.5) [42]. The IR effect on the primary metrics is therefore real but modest in magnitude, consistent with the substantially overlapping distributions visible in Figure 5.1.

5.3.3 Disaggregated Analysis: Path B vs. Path A and Path C vs. Path A

To determine whether both IR types independently contribute to the pooled effect, Table 5.6 reports separate Wilcoxon signed-rank tests for each IR path against the no-IR baseline. Both Path B and Path C achieve corrected significance on BC ($p_{\text{adj}} = 0.0004$ and 0.0188, respectively), confirming that each IR variant in-

dependently improves behavioral completeness. On RT, however, an asymmetry emerges: Path B's improvement survives correction ($p_{\text{adj}} = 0.0008$, $\delta = +0.368$) while Path C's does not ($p_{\text{adj}} = 0.158$, $\delta = +0.124$), despite Path C having the highest mean RT in Table 5.4. Neither path shows a significant effect on SC or VC, consistent with the pooled analysis.

Table 5.6: RQ1 disaggregated comparisons.

Comparison	**BC**	**RT**	**SC**	**VC**
B vs. A — p	0.0001	0.0002	0.161	0.733
B vs. A — p_{adj}	0.0004*	0.0008*	0.645	1.000
B vs. A — Cliff's δ	+0.219	+0.368	+0.105 (n.s.)	+0.014 (n.s.)
C vs. A — p	0.0047	0.0396	0.176	0.879
C vs. A — p_{adj}	0.0188*	0.158	0.705	1.000
C vs. A — Cliff's δ	+0.155	+0.124 (n.s.)	+0.129 (n.s.)	+0.123 (n.s.)

*Significant after Bonferroni correction at family-wise $\alpha = 0.05$.

5.3.4 Triangulation: GT-SC against Human-Authored Tests

GT-SC is a secondary triangulation metric scored against the human-authored test scripts (Section 4.6.2). Table 5.7 reports per-path central tendency, and the count of requirements for which the generated test reaches at least half coverage and full coverage of the human-authored union.

Table 5.7: GT-SC across Paths A, B, C ($n = 31$ requirements). Mean and median are over per-requirement values.

Path	**Mean**	**Median**	≥ 0.50	$= 1.00$
A (No-IR)	0.500	0.500	14	1
B (Structured IR)	0.561	0.550	16	2
C (Executable IR)	0.617	0.674	19	5

5.4 RQ2: Structured vs. Executable IR

RQ2 isolates the contribution of executability and self-validation by comparing Path C (executable xSpec with mandatory scenario validation) directly against Path B (structured JSON with schema validation).

5.4.1 Primary Metric Comparison

Table 5.8: Path C vs. Path B primary metrics ($n = 31$ paired requirements).

Metric	Path B mean	Path C mean	Median diff. (C−B)
BC	0.863	0.906	0.000
RT	0.823	0.895	+0.032
SC	0.986	0.994	0.000
VC	0.990	0.991	0.000

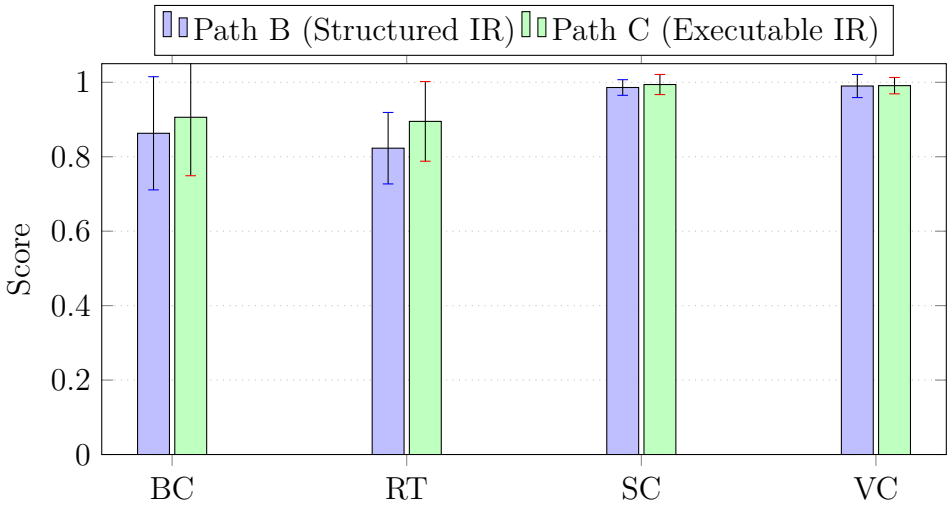


Figure 5.2: RQ2 primary-metric comparison: Path B vs. Path C (means with SD error bars, from Table 5.4).

Table 5.9: Wilcoxon signed-rank tests for Path C vs. Path B ($k = 4$). Bonferroni-adjusted values use $p_{\text{adj}} = \min(1,\ p \cdot k)$; significance is evaluated at $p_{\text{adj}} < 0.05$.

Metric	W	p	p_{adj}	Significant?	Cliff's δ
BC	57.5	0.223	0.890	No	−0.073 (n.s.)
RT	90.0	0.0034	0.0135*	Yes	+0.246
SC	7.0	0.893	1.000	No	+0.028 (n.s.)
VC	74.0	0.616	1.000	No	−0.137 (n.s.)

*Significant after Bonferroni correction ($p_{\text{adj}} < 0.05$).

Across BC, SC, and VC, Paths B and C are statistically indistinguishable: none of the Bonferroni-corrected p-values approaches significance, so the corresponding Cliff's δ values are not interpreted. The only metric on which the executable IR significantly outperforms the structured IR is requirement traceability (RT, $p_{\text{adj}} = 0.0135$). Because an effect size is meaningful only when the underlying test detects a difference, RT is the sole comparison whose effect size we interpret—and even there the effect is *small* ($\delta = +0.246$, below the 0.33 small/medium boundary of Section 4.6.5). In practical terms, adding executability and scenario validation on

top of a structured IR yields a measurable but modest gain confined to how faithfully generated behaviors trace back to the requirement, with no detectable advantage in completeness, signal correctness, or value correctness.

5.4.2 Within-Path Variance: Output Stability Across Runs

Beyond central tendency, the run-to-run stability of each path is itself a quality signal: a path that produces widely varying output across identical runs is harder to trust in practice. Table 5.10 reports, for each metric, the standard deviation across the three runs averaged over the 31 requirements (lower is more stable).

Two patterns stand out. First, both IR paths are more stable than direct generation on the two metrics that vary most—BC and SC—with the standard deviation on SC falling from 0.030 (Path A) to 0.008 (Path B) and 0.006 (Path C); structuring the reasoning, in either IR form, makes signal selection markedly more consistent run-to-run. Second, the two IR paths are essentially indistinguishable in stability (their per-metric SDs differ by at most 0.005), mirroring the central-tendency finding that executability adds little beyond structuring on these metrics. The small VC standard deviation for Path C (0.010 vs. 0.005 for Paths A/B) reflects a handful of value-formatting differences rather than a systematic instability. Overall, introducing an IR not only raises mean quality but also tightens the distribution of outcomes, which is the more decision-relevant property for industrial adoption.

Table 5.10: Within-path variance: standard deviation across the three runs, averaged over the 31 requirements (lower indicates more stable generation).

Metric	Path A SD	Path B SD	Path C SD
BC	0.068	0.047	0.046
RT	0.067	0.059	0.064
SC	0.030	0.008	0.006
VC	0.005	0.005	0.010

5.5 RQ3: Augmentation of Existing Tests

RQ3 evaluates the report-only Path D pipeline on the 31-requirement dataset. For every requirement, the highest-BC Path C run was selected as the candidate, and an LLM analyst compared it against the union of the human-authored linked tests for that requirement. The analyst emitted a structured report (gap / preserved / lost / agreement classifications per canonical expected behavior); no merged or rewritten test script is generated. The comparison metric (ΔBC) and the descriptive counts in this section come from those reports together with the Phase 2 evaluator scoring of the human-authored union.

5.5.1 Path D Dataset

All 31 requirements in the dataset have at least one linked human-authored test; the dataset is therefore fully eligible for Path D in principle. One handle's existing-test union exceeded the SDK's per-call timeout window during Phase 2 scoring and is excluded from the BC-based analyses, leaving 30 handles for the ΔBC analysis and 29 with usable analyst reports (one further handle was reported by the analyst as `evaluation_missing` due to a transient read failure). Counts are summarized in Table 5.11.

Table 5.11: Path D dataset eligibility. "SDK timeout" refers to the Claude Agent SDK's request-timeout limit reached during Phase 2 scoring of the human-authored union; "analyst transient" refers to a single analyst-side read failure on a different handle.

Status	Count
Test retrieved & union scored	30
Test retrieved but Phase 2 scoring timed out (SDK timeout)	1
Test linkage missing	0
Eligible for ΔBC analysis	30
With usable analyst report (gap / preservation classification)	29

5.5.2 Gap Identification: GP and GR

Across the 29 handles with usable analyst reports, the analyst identified 96 candidate gaps (mean ≈ 3.3 per requirement). Reliability numbers (Table 5.12) are computed against an LLM-as-judge second rater rather than a human one,[1] so the resulting GP and GR are LLM-vs-LLM agreement numbers.

5.5.3 Preservation Rate (PR)

Path D as evaluated here is report-only — the pipeline does not emit a merged test, so the "replaced" and "annotated" outcomes from the original methodology do not apply. Preservation is instead measured at the canonical-behavior level: of the 113 expected behaviors that the human-authored union covers (sum of `preserved` and `lost` entries across the 29 reports), 107 are also covered by the highest-BC Path C run, while 6 are covered by the human test but missing from the candidate.

$$\mathrm{PR} = \frac{107}{113} = 0.947$$

A breakdown of preservation outcomes is given in Table 5.13.

[1] A separate Claude Sonnet session was given the requirement text, the discovery handle, the existing-test union, the highest-BC Path C candidate, and the analyst's gap list, without access to the analyst's preserved/lost/agreement classifications. It produced a per-gap TP/FP verdict on the analyst's 96 candidates and an independent rater gap set; the orchestrator computed Gap Precision against the per-gap verdicts and Gap Recall against the rater set.

Table 5.12: Gap identification: descriptive counts and reliability against the LLM-as-judge second rater. "Genuine gaps (per rater)" is the size of the judge's independent gap set, computed without access to the analyst's classification.

Quantity	**Count**
Identified gaps (by analyst)	96
confirmed genuine (TP)	84
false alarms (FP)	12
Genuine gaps (per rater)	133
identified (TP)	84
missed (FN)	49
Gap Precision (GP)	0.875
Gap Recall (GR)	0.632

Table 5.13: Preservation outcomes at the expected-behavior level. The "Replaced" and "Annotated" outcomes from the original methodology do not apply because Path D as evaluated here does not emit a merged test.

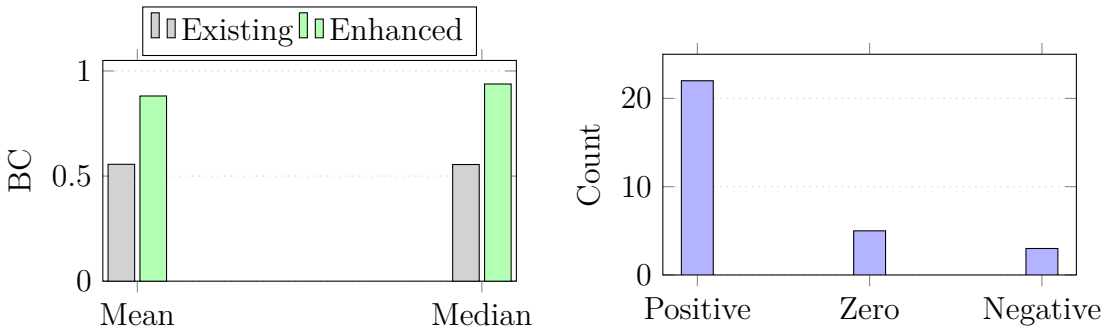
Outcome	**Count**
Preserved (covered by both human and Path C candidate)	107
Lost (covered by human, missing from candidate)	6
Total human-covered expected behaviors	113

5.5.4 Coverage Improvement (ΔBC)

For each of the 30 eligible requirements, BC was computed for the union of the human-authored linked tests ($BC_{existing}$) and for the highest-BC Path C run ($BC_{enhanced}$). Results are shown in Table 5.14.

Table 5.14: Behavioral completeness before and after taking the highest-BC Path C run as the enhanced test ($n = 30$).

	Mean	**Median**	**SD**
$BC_{existing}$	0.556	0.555	0.324
$BC_{enhanced}$	0.881	0.938	0.139
ΔBC	+0.326	+0.302	0.316



(a) Aggregate BC before and after enhancement.

(b) Sign distribution of paired ΔBC values ($n = 30$).

Figure 5.3: RQ3 coverage-improvement summary from reported aggregate values and paired-difference counts.

A Wilcoxon signed-rank test on the 30 paired differences yielded $W = 14.0$, $p = 6.5 \times 10^{-5}$, Cliff's $\delta = +0.587$ (large effect; sign convention δ(enhanced − existing)). Of the 30 paired differences, 22 are positive, 5 are zero, and 3 are negative; the negative cases occur on requirements where $BC_{existing} \geq 0.75$. The public appendix reports an audit summary of these paired outcomes (Appendix A.3).

5.5.5 Reference-Run Sensitivity

To assess sensitivity to which of the three Path C runs is taken as the reference, one requirement (`x0400000028DE7EA6`, chosen because it had the largest run-to-run BC range in Path C: 0.643/0.786/0.429) was re-analyzed with each of the three Path C runs as the reference. The resulting gap sets are compared in Table 5.15; the gap-set overlap with run 1 falls from 75.0% (run 2) to 12.5% (run 3), and the rater-confirmed TP count varies across 8/9/4. Implications for the choice of run-selection rule are taken up in Chapter 6.

Table 5.15: Reference-run sensitivity for handle `x0400000028DE7EA6`. "Genuine gaps (TP)" uses the same LLM-as-judge second rater applied to the GP/GR step earlier in this section (Table 5.12).

Reference run	Gaps identified	Genuine gaps (TP)	Overlap with run 1
Run 1	8	8	100.0%
Run 2	10	9	75.0%
Run 3	5	4	12.5%

5.5.6 Qualitative Analysis of Gap and Preservation Types

Each entry in the analyst reports was assigned exactly one of seven mutually exclusive categories (per Step 4 of the analyst prompt). Distribution across the 29 reports is shown in Table 5.16; counts are over individual classified entries (96 gaps, 98 categorized preserved entries, and 6 lost entries — 9 preserved entries were emitted by the analyst as bare ids without categorization and are excluded from the per-category breakdown).

Table 5.16: Distribution of identified gaps, preserved behaviors, and lost behaviors by category. Percentages are within column.

Category	Gaps (%)	Preserved (%)	Lost (%)
Nominal behavior	12.5 (12/96)	38.8 (38/98)	0.0 (0/6)
Boundary values	3.1 (3/96)	5.1 (5/98)	0.0 (0/6)
Invalid inputs	5.2 (5/96)	1.0 (1/98)	16.7 (1/6)
Fault conditions	62.5 (60/96)	29.6 (29/98)	83.3 (5/6)
State persistence	0.0 (0/96)	4.1 (4/98)	0.0 (0/6)
Timing constraints	2.1 (2/96)	1.0 (1/98)	0.0 (0/6)
Mode interactions	14.6 (14/96)	20.4 (20/98)	0.0 (0/6)

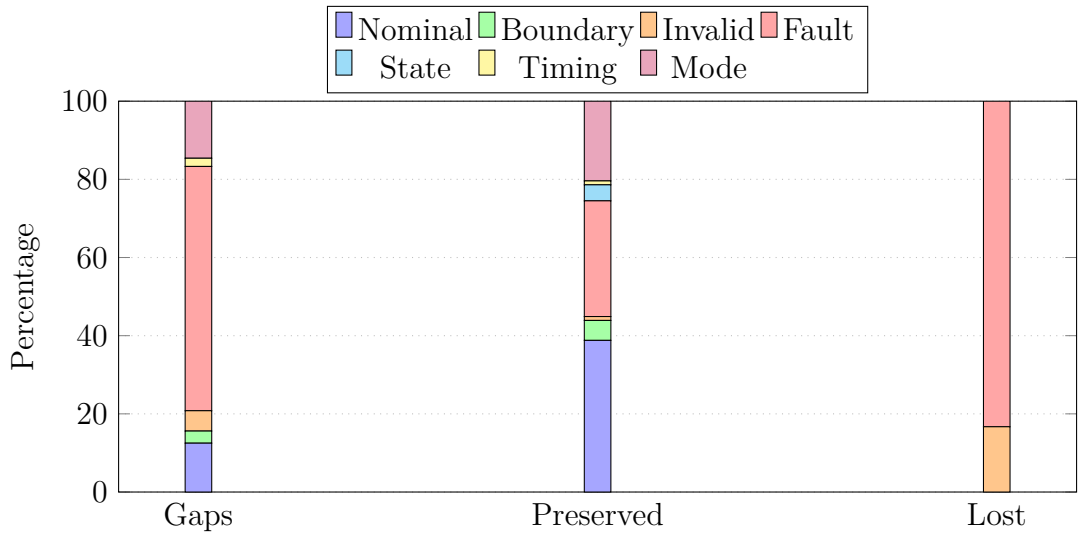


Figure 5.4: Category distribution of identified gaps, preserved behaviors, and lost behaviors (percentages from Table 5.16).

6

Discussion

This chapter interprets the empirical findings reported in Chapter 5, connects them to the research questions, and evaluates their methodological and practical implications. The discussion is structured as follows. Section 6.1 interprets RQ1–RQ3 in sequence. Section 6.2 discusses cross-cutting patterns across the metrics and triangulation signals. Section 6.3 positions the findings against the related-work baseline. Section 6.4 outlines implications for industrial deployment. Finally, Section 6.5 consolidates the threats to validity for the whole study in a single location.

6.1 Research-Question Interpretation

6.1.1 RQ1: Effect of Using an Intermediate Representation

RQ1 asks whether introducing an intermediate representation improves test generation quality relative to direct generation. The principal finding is a *mixed but meaningful* effect profile.

On behavioral completeness (BC), the pooled IR comparison shows a statistically significant improvement over direct generation by the paired median criterion (Table 5.5, median IR − A = +0.036, $p_{\text{adj}} = 0.0024$). The disaggregated tests further indicate that both IR variants contribute to this effect (Table 5.6; B vs. A: $p_{\text{adj}} = 0.0004$, C vs. A: $p_{\text{adj}} = 0.0188$). This supports the central premise that forcing an intermediate decomposition stage helps the generator recover requirement-mandated behaviors that direct generation omits.

On requirement traceability (RT), the pooled comparison also shows a statistically significant improvement (Table 5.5, median IR − A = +0.052, $p_{\text{adj}} = 0.0040$, Cliff's $\delta = +0.273$). Mean RT values increase monotonically from Path A to Path C (0.803, 0.823, 0.895; Table 5.4), confirming that IR-based generation yields more traceable outputs. The disaggregated analysis, however, reveals an asymmetry: Path B's RT improvement over Path A survives Bonferroni correction (a standard adjustment for testing multiple metrics simultaneously; $p_{\text{adj}} = 0.0008$), whereas Path C's does not ($p_{\text{adj}} = 0.158$), despite Path C having the highest mean RT. This is most likely a statistical power limitation rather than evidence of no effect: Path C's pre-correction p-value (0.0396) indicates a real trend that does not sur-

vive the conservative Bonferroni adjustment at $n = 31$, and Path C exhibits higher cross-requirement RT variance than Path B (SD 0.064 vs. 0.059; Table 5.10), which reduces the Wilcoxon test's sensitivity to the shift in central tendency.

Importantly, BC and RT are complementary rather than redundant: BC is requirement-centered (fraction of required behaviors covered), while RT is generation-centered (fraction of generated behaviors that are justified by the requirement). Both improving simultaneously in the pooled comparison indicates that IR-guided generation recovers more requirement-mandated content without proportionally increasing unjustified content.

Signal correctness (SC) and value correctness (VC) do not show statistically meaningful path effects in the RQ1 inferential tests. This is an important boundary condition: the main measurable gain from IR use appears in requirement-level behavioral coverage rather than lexical/syntactic signal-value fidelity.

The reason for this asymmetry lies in what the IR actually controls. BC and RT measure whether the LLM *selects the right behaviors* from the requirement and *justifies them*—decisions that directly benefit from a structured decomposition step. SC and VC, by contrast, measure whether signal names and enumeration values are transcribed correctly. These surface-level facts are supplied by the discovery JSON, which all three paths share identically; the generation strategy does not change the input vocabulary available to the generator. This pattern aligns with Ma et al.'s characterization of LLMs as "semantic giants but computational dwarfs" [30]: the IR helps the model organize *which* behaviors to test (a semantic task), but does not improve its ability to compute precise boundary values or signal encodings (a computational task). Hence the effect profile is mixed by design, not by failure: the IR addresses the bottleneck where it matters - behavioral decomposition, while leaving signal-level fidelity to the discovery instrument.

6.1.2 RQ2: Structured IR Versus Executable IR

RQ2 isolates the incremental value of executability and scenario validation by comparing Path C against Path B. The result pattern is narrow and specific.

Across BC, SC, and VC, the two IR types are statistically indistinguishable (Tables 5.8 and 5.9). The only metric with a corrected significant difference is RT, where Path C outperforms Path B (median C − B = +0.032, $p_{\text{adj}} = 0.0135$, a small effect, $\delta = +0.246$)..

Methodologically, this suggests that *structuring* provides most of the measurable benefit already, while *executability* adds a more targeted gain in requirement traceability. This targeted gain is plausible: executable IR validation constrains internally inconsistent behavioral plans before script generation, potentially reducing loosely justified expansions.

The within-path variance profile supports a stability interpretation. Both IR paths

reduce BC and SC run-to-run variation compared to direct generation (Table 5.10; BC SD: 0.068 for A versus 0.047/0.046 for B/C; SC SD: 0.030 for A versus 0.008/0.006 for B/C). The practical implication is that introducing either of the two IR types studied here act as a stability lever; executability adds value, but not as a universal across-metric uplift.

The RT-specific advantage can be traced to concrete mechanisms in the xSpec framework (Appendix A.5). First, encoding requirement semantics as a Python state machine forces the LLM to declare explicit states and transitions rather than describing behavior in free-form prose. Automotive requirements are inherently stateful-mode-dependent behavior, fault transitions, timer-guarded recovery and the state-machine representation aligns the generated artifact with this structure. Second, xSpec's edge-tracking utilities (`_check_rising_edge`, `_check_falling_edge`) require the LLM to distinguish "signal *is* value V" from "signal *changed to* value V." Natural-language requirements often conflate the two; the executable IR makes the distinction mandatory, anchoring each behavior in a verifiable transition that maps to a traceable requirement passage. Third, xSpec's scenario runner acts as a semantic gate: declared behaviors must compile *and* pass execution before the artifact proceeds to script generation. Path B's JSON schema validates structure but cannot catch logical contradictions - an impossible state transition or a timer that never fires will pass schema validation but fail scenario execution. This self-correcting loop filters out loosely justified content that would otherwise inflate the untraceable block count. The choice of Python over formal notations (CTL, SysML) is deliberate: Yu et al. report only ∼62% semantic precision when LLMs generate SysML v2 models, attributing the shortfall to a "syntax cliff" where the notation's verbosity exceeds what current models can reliably produce [27]. Python stays in the LLM's high-competence zone while still providing executability - a pragmatic trade-off between formal rigor and generation reliability.

6.1.3 RQ3: Augmentation of Existing Human-Authored Tests

RQ3 evaluates whether the report-only enhancement pipeline can identify meaningful omissions while preserving existing useful content. The overall result is strongly positive for augmentation utility.

On paired requirement-level completeness, the enhancement candidate substantially exceeds the existing-test union (Table 5.14; median $\Delta\text{BC} = +0.302$, Wilcoxon $W = 14.0$, $p = 6.5 \times 10^{-5}$, Cliff's $\delta = +0.587$). The sign distribution (23 positive, 4 zero, 3 negative) indicates that gains are not driven by a small number of outliers.

Illustrative example. Handle `28B31956` (battery-voltage low-pass filter, medium complexity) illustrates the effect concretely. The requirement specifies an IIR filter with two configurable parameters - filter slope (`X1BK0`, range −40 to −20 dB/decade) and cutoff frequency (`X1BKZ`, range 0–150 rad/s) plus fault-handling for several error states. The evaluator extracted six expected behaviors ($|E_{\text{req}}| = 6$); the existing human-authored test covered three: nominal IIR filtering, `ErrorIndicator` propagation, and `Spare` fault handling ($\text{BC}_{\text{ex}} = 0.583$). The Path C candidate covered

all six ($BC_{enh} = 1.000$, $\Delta BC = +0.417$), flagging three gaps:

1. *NotAvailable fault propagation*—the existing script partially exercised this path but used an ambiguous intermediate value; the generated test injected the canonical raw 255 and verified propagation to filtered 511.

2. *Filter-slope boundary*—`X1BK0` at -20 dB/decade (upper range limit), never set by the existing script.

3. *Cutoff-frequency boundary*—`X1BKZ` at 0 rad/s, verifying that gain k=0 freezes the output, also never exercised.

One gap is a fault-condition behavior (*NotAvailable* fault propagation) and two are configuration-boundary behaviors, consistent with the aggregate category profile (Table 5.16), where fault and boundary conditions account for the majority of identified gaps.

The three preserved behaviors were retained without loss (PR = 1.0 for this handle), confirming that the pipeline augments rather than displaces existing coverage.

Preservation and gap-quality metrics further characterize this effect. Preservation rate is high (PR = 0.947), indicating that the candidate generally retains coverage represented in the human-authored tests. Gap precision is strong (GP = 0.875), while gap recall is moderate (GR = 0.632), suggesting the analyst is more conservative than exhaustive: flagged gaps are often valid, but a non-trivial fraction of genuine gaps remain unflagged.

The category profile provides additional interpretive depth. Most identified gaps are fault-condition behaviors (62.5% in Table 5.16), whereas preserved content is more distributed. This aligns with a practical view in which existing suites are stronger on nominal pathways than on fault-mode breadth.

Finally, the reference-run sensitivity check (Table 5.15) shows that high-variance requirements can materially change gap sets depending on the selected Path C run (run-1 overlap: 75.0% for run 2, 12.5% for run 3). This does not invalidate the augmentation result, but it does imply that production workflows should avoid single-run dependence for high-variance cases.

6.2 Cross-Cutting Interpretation

6.2.1 BC Versus GT-SC: Complementary, Not Redundant

A central cross-cutting pattern is the persistent gap between requirement-centric BC and human-test-centric GT-SC (Section 5.3). BC levels are materially higher than GT-SC across all paths. This divergence is expected under the dual-oracle design (Section 4.6.2): BC rewards alignment with the requirement specification, while GT-SC rewards overlap with existing engineering artifacts.

Interpretively, this gap is not a contradiction but a complementary-strengths signal. Existing human-authored scripts encode domain engineering knowledge that the generation pipeline cannot easily replicate: platform-specific timing calibrations, test-bench wiring conventions, and signal-conditioning patterns refined through iterative validation cycles. These are competencies acquired through deployment experience rather than derivable from the requirement text alone, and they explain why generated tests do not score higher on GT-SC despite strong BC. Conversely, the RQ3 results demonstrate that existing suites systematically undercover requirement-mandated behaviors, particularly fault-condition and boundary-value scenarios (Table 5.16; 62.5% of identified gaps are fault conditions). Handle `28B31956` illustrates this asymmetry concretely: the human script exercised nominal IIR filtering with domain-appropriate signal conditioning, yet omitted one fault-condition behavior (*NotAvailable* fault propagation) and two configuration-boundary behaviors (`X1BK0` at $-20\,\mathrm{dB/decade}$, `X1BKZ` at $0\,\mathrm{rad/s}$) that the generated candidate caught. This pattern (domain craft on one side, requirement-intent coverage on the other) is consistent with Pan et al.'s finding that requirement-level coverage exposes faults that code-coverage-driven selection systematically misses [36], and it is precisely the setting where augmentation has higher practical value than replacement.

6.2.2 Instrument-Reliability Effects on Interpretation Confidence

The manual validation study (Section 5.2) provides a necessary confidence calibration.

BC extraction quality is relatively strong on precision (0.923 for both raters) but lower on recall (0.789 and 0.800). Because the recall falls below the 0.85 reliability threshold defined in Section 4.6.4, BC is officially instrument-limited by the study's own criteria. The practical consequence is that the evaluator agent misses approximately 20% of the expected behaviors that human raters identify, which deflates the denominator of the BC formula and therefore inflates the reported absolute BC scores. For example, if a requirement has 10 true expected behaviors but the evaluator extracts only 8, a generated test covering 6 of those 8 would score BC = 0.75, whereas the correct value against the full set would be BC = 0.60.

This inflation affects all paths equally only if the missed behaviors are path-independent - that is, if the evaluator's extraction failures are driven by requirement-side ambiguity rather than by properties of the generated test. The within-subjects design provides a structural argument for this assumption: the evaluator extracts expected behaviors from the requirement text and discovery artifact, both of which are identical across paths. The generated test influences only the coverage classification (covered, partial, missing), not the behavior extraction step. Hence, while absolute BC values should be treated as upper-bound estimates, the *relative* differences between paths, which drive the RQ1 and RQ2 conclusions—are protected from this bias because the same (potentially incomplete) behavior set serves as the denominator for all paths on the same requirement.

The remaining metrics (RT, SC, VC) were not independently validated against human raters; their absolute values should be treated as instrument-conditioned, while relative path comparisons are informative because any systematic bias is constant across paths.

A separate instrument limitation applies to the RQ3 gap metrics. Gap Precision (0.875) and Gap Recall (0.632) are computed against an LLM-as-judge second rater rather than a human one, making them measures of LLM-vs-LLM agreement. Zheng et al. show that strong LLMs can achieve high concordance with human judges on many evaluation tasks [40], but LLM-vs-LLM agreement may systematically overestimate or underestimate reliability compared to human judgement, particularly on domain-specific gap classifications where the LLM and the analyst share similar failure modes. The GP and GR values should therefore be treated as descriptive agreement indicators rather than human-validated accuracy scores. Importantly, the qualitative findings from RQ3 - the dominance of fault-condition gaps (62.5%), the high preservation rate (94.7%), and the significant coverage improvement (ΔBC)—do not depend on the GP/GR instrument, as ΔBC is computed from the requirement-centric evaluator validated in Section 5.2.

6.3 Positioning Against Related Work

Relative to IR-guided generation studies reviewed in Chapter 3, we contribute a controlled same-dataset comparison across IR formality levels under a shared pipeline. The findings both confirm and refine the literature narrative.

Code representation improves behavioral coverage. Liao et al. report ∼30% branch-coverage improvement when unit-test generation is guided by Jimple intermediate code and control-flow graphs [33]. Han et al. show that semantic skeletons (Subject–Verb–Object–Polarity triplets) reduce assertion hallucination to 0.183 [32]. Our BC improvement from Path A to Paths B/C confirms this pattern in a different domain: representing requirement semantics in a structured form—whether JSON or Python—helps the LLM recover behaviors that direct generation omits. The thesis applies a similar IR-guided approach to automotive system-level test generation, where the "code" is a signal-orchestration script rather than a function-under-test wrapper, though the domain, language, and toolchain differ substantially from those studied settings.

Executable IR provides targeted, not universal, gains. Formal verification approaches (Zrelli et al.'s CTL translations [34]; Thakur et al.'s B-Method specifications [35]) can prove safety properties but do not produce executable HIL test scripts—an "executability gap" between proof and deployment. At the other extreme, Yu et al. find only ∼62% semantic precision when LLMs generate SysML v2 models, attributing the shortfall to notation verbosity that exceeds current model capabilities [27]. xSpec occupies a pragmatic middle ground: Python provides sufficient executability for scenario-level self-checking while remaining in the LLM's zone of high generation reliability. Our RQ2 result - a significant RT advantage

but no BC/SC/VC difference refines the narrative that executability is universally superior. The gain is real but narrow: the execution checkpoint filters logically inconsistent plans, improving traceability without changing the discovery-dependent surface-level metrics.

Augmentation over replacement. Danglot et al.'s DSpot [43] and Tufano et al. [44] demonstrate that amplifying existing tests, adding assertions, mutating inputs - is a viable alternative to full regeneration. Pan et al.'s RTM framework [36] further shows that requirement-coverage-guided test selection yields significantly higher fault-detection rates than code-coverage-based selection alone. Our RQ3 results contribute this line of work: the gap-report workflow achieves augmentation with 87.5% precision, and the dominance of fault-condition gaps (62.5%) aligns with Pan et al.'s finding that requirement-level coverage exposes behaviors that code-coverage metrics systematically miss. The practical implication is the same: existing suites are strongest on nominal paths; LLM-guided augmentation adds most value at the fault and boundary periphery.

Evaluation reliability in context. Ma et al. characterize LLMs as "semantic giants but computational dwarfs" [30]—strong at parsing intent but weak at boundary-value computation. Siddiq et al.'s ULT/PLT benchmarks [31] show that high leaderboard performance can be driven by memorization of public code rather than genuine reasoning, with substantial drops on novel, time-stamped problems. Both findings contextualize the present study. First, the IR's failure to improve SC/VC is expected: the computational-dwarf limitation applies to precise value transcription regardless of decomposition strategy. Second, the proprietary, unseen nature of the Volvo requirements mitigates the memorization concern, there is no public training-set overlap for these specifications, but reinforces the importance of the manual validation study as a reliability anchor.

6.4 Implications for Industrial Adoption

The evidence supports a staged adoption strategy rather than all-at-once replacement.

First, introducing an IR layer is justified even when executability is not yet feasible for every requirement class. The RQ1 and variance results show that structured decomposition alone already improves behavioral outcomes and stability.

Second, executable IR should be prioritized where traceability discipline and downstream analysis reuse matter most. The RT improvement in RQ2 and the Path D dependence on executable references both point in this direction.

Third, the Path D results support a human-in-the-loop augmentation workflow: use analyst-reported gaps as review candidates, preserve existing validated intent, and target additional fault-handling coverage. Under this model, engineers shift from full-script authorship to higher-leverage adjudication tasks.

Recent industrial evidence corroborates this staged view. Wang et al. show that decomposing a manual automotive test process into discrete LLM-automated steps while preserving the existing workflow structure can fully replace 2–3 FTE of manual effort for vehicle API testing [45]. Their success with structured API specifications suggests that the benefits of IR-guided decomposition observed in our RQ1 results may extend to broader process-level automation when combined with well-defined intermediate artifacts.

6.4.1 Cost and Efficiency

Table 6.1 summarises the total and per-run API costs and wall-clock times across the three generation paths, using the model configuration described in Section 4.6.1.

Table 6.1: Cost and efficiency metrics across IR paths ($n = 93$ runs per path).

Path	Total Cost	Total Time	Runs	Avg Cost/Run	Avg Time/Run
A (Direct)	\$908.33	424.1 h	93	\$9.56	267.9 min
B (Structured)	\$1018.22	436.1 h	93	\$10.95	281.3 min
C (Executable)	\$1110.93	445.8 h	93	\$11.82	284.6 min
Total	\$3037.48	1306.0 h	279	\$10.88	278.4 min

Cost increases monotonically with IR formality: direct generation (Path A) is the least expensive at \$9.56 per run, structured IR (Path B) costs \$10.95, and executable IR (Path C) is the most expensive at \$11.82—a 23.6% increase from Path A to Path C. This overhead reflects the additional token expenditure for IR generation and, in the case of Path C, mandatory scenario validation with potential retries. Wall-clock times are similar across paths (267.9–284.6 minutes per run), indicating that IR formality does not substantially prolong execution; the additional computation is absorbed within the model's token budget rather than adding sequential delay.

These measurements provide an empirical basis for assessing the resource trade-offs of IR-guided generation. The \$2.26 additional cost per run for Path C relative to Path A (\$202.60 across all 93 runs) must be weighed against the quality improvements reported in Chapter 5: the significant gains in behavioral completeness and requirement traceability (RQ1), the targeted traceability advantage of executability (RQ2), and the augmentation utility of the executable IR for gap detection (RQ3). In contexts where a single missed fault-condition behavior can trigger costly field failures or safety-case rework, the marginal cost of the executable IR may be justified, though this study evaluated static quality metrics only and did not measure dynamic fault detection on SIL/HIL rigs.

Operationally, two safeguards are indicated by the data:

- Use multi-run or consensus reference selection for high-variance requirements before gap analysis.

- Track instrument-health indicators (especially BC extraction precision and recall) as part of regular pipeline maintenance.

6.5 Threats to Validity

This section consolidates all threats to the validity of the study in a single location, spanning the dataset, the evaluation instruments, and the broader research design. For each category it states the threat, reassesses it in light of the observed results, and incorporates the practical limitations that bound the scope of the findings. We use the standard categories of construct, internal, external, and conclusion validity, followed by reliability and replicability.

6.5.1 Construct Validity

Construct validity concerns whether the metrics measure what they purport to measure. The primary metrics (BC and RT) are computed by an LLM-based requirement-centric evaluator that extracts expected behaviors from the requirement text and classifies coverage and traceability. Because this evaluator is itself an LLM-as-a-judge instrument rather than a human rater, its reliability is a direct construct-validity threat: systematic extraction errors would bias the metrics. The manual validation study (Section 5.2) bounds this threat for BC extraction, which reached high precision (0.923) but borderline recall (0.789/0.800, below the 0.85 target). The practical consequence is that the evaluator misses roughly 20% of human-identified behaviors, deflating the BC denominator and inflating absolute BC scores; absolute BC should therefore be read as an upper-bound estimate. RT, SC, and VC were not independently validated against human raters and should likewise be treated as instrument-conditioned. Crucially, because the behavior-extraction step depends only on the requirement and discovery artifacts identical across paths, any such bias is constant across paths, so the *relative* comparisons that drive the RQ1 and RQ2 conclusions are protected even where absolute values are not.

A second LLM-as-a-judge instrument underlies the RQ3 gap metrics: Gap Precision (0.875) and Gap Recall (0.632) are measured against an independent LLM rater rather than a human, and are therefore LLM-vs-LLM agreement figures that may over- or under-state reliability, particularly where the analyst and the judge share failure modes [40]. The substantive RQ3 conclusions—the dominance of fault-condition gaps, the 94.7% preservation rate, and the significant ΔBC—do not rely on GP/GR, as ΔBC is computed by the validated requirement-centric evaluator. A related, unvalidated construct is the "potential novel coverage" recorded when generated behaviors exceed the human-authored union: these are noted but not independently confirmed as correct.

6.5.2 Internal Validity

Internal validity concerns whether observed differences are caused by the manipulated variable (the IR) rather than by confounds. The within-subjects design is the

principal safeguard: every requirement is processed through all paths, sharing one discovery phase, one generator, one output DSL, and one model family, so differences are attributable to the IR rather than to input data or tooling. Two residual threats remain. First, LLM nondeterminism introduces run-to-run variance; we address it with three runs per configuration, but the RQ3 reference-run sensitivity analysis (Table 5.15) shows that high-variance requirements can still yield materially different gap sets depending on which Path C run is chosen as the reference production workflows should therefore avoid single-run dependence for such cases. Second, the evaluator and the generators are drawn from the same model family (Claude), which could in principle introduce a shared bias; the within-subjects control mitigates this for path comparisons but cannot fully exclude it for absolute values.

6.5.3 External Validity

External validity concerns generalizability. The evaluation is deliberately scoped to a single functional domain (one Logical Design Component in the CCIOM body/cabin electronics subsystem), a single organization and systems-engineering tool, a single proprietary test DSL, and a single LLM family. The 31 requirements also share a near-constant external signal count, which limits within-dataset signal-complexity variation. The findings are therefore most defensible for similar automotive requirement/test ecosystems and should be transferred cautiously beyond that envelope. Whether the IR benefit generalizes across domains, platforms, test formats, and model families is left to future work (Section 7.3).

6.5.4 Conclusion Validity

Conclusion validity concerns whether the statistical inferences are sound. With $n = 31$ paired requirements, statistical power is limited: Bonferroni correction across the four primary metrics lowers the per-family threshold and can cause real but modest effects to miss corrected significance as plausibly occurs for Path C's RT effect in the disaggregated analysis. We mitigate this by reporting effect sizes and exact p-values alongside corrected values, by interpreting an effect size only where the underlying test is significant, and by treating non-significant results as inconclusive rather than as evidence of no effect. We also note explicitly that the significant RQ1 and RQ2 effects are *small* in magnitude (Sections 5.3 and 5.4). The three runs per configuration characterize but do not eliminate nondeterminism, and may under-sample the tail of the output distribution.

6.5.5 Reliability and Replicability

Reliability concerns whether the study can be reproduced. All experiments use commercial Claude models accessed through Anthropic's API. Unlike open-weight models (e.g., Llama, Mistral), where a specific checkpoint can be downloaded and executed indefinitely, commercial API-served models may be silently updated, retrained, or deprecated by the provider without changing the public version identifier, so the exact snapshots used here (Section 4.6.1) may not be available for future replication. We mitigate this by recording full model identifiers and API parameters,

caching all discovery artifacts for reuse across runs, and reporting per-run variability so that readers can assess nondeterminism bounds. A further, more mundane reliability limitation is instrumentation: first-attempt versus retried xSpec validation counts were not persisted in this version of the pipeline, preventing analysis of the executable IR's validation-loop cost; this is addressed as instrumentation work in Section 7.3. Exact numerical reproduction is therefore not guaranteed, and the results should be read as characterizing the IR-strategy effect under a specific model generation rather than as immutable constants.

Cutting across all of the above categories is a deliberate scope limitation: all metrics are computed by *static* analysis of generated scripts. No generated test was executed on a SIL or HIL rig, so dynamic properties - timing behavior, race conditions, and hardware-interaction fidelity - remain unverified. Closing this static-to-dynamic gap is the foremost item of future work (Section 7.3).

6.6 Discussion Summary

To summarize the key interpretations:

- **RQ1:** In this study, introducing an intermediate representation significantly improves both behavioral completeness and requirement traceability relative to direct generation. Signal and value correctness, already high for direct generation (≥ 0.88), do not change significantly with IR use. Both IR types show BC improvement over the baseline; of the two, only Path B's individual RT effect survives Bonferroni correction at $n = 31$.

- **RQ2:** Structured and executable IRs are statistically indistinguishable on BC, SC, and VC. The executable IR provides a targeted, significant advantage on requirement traceability and reduces output variance on BC and SC. The practical implication is that structuring the generation process delivers most of the measurable benefit, while executability adds targeted value.

- **RQ3:** The report-only enhancement pipeline demonstrates strong augmentation utility: the candidate test preserves 94.7% of existing behaviors while identifying gaps with 87.5% precision. Most identified gaps are fault-condition behaviors (62.5%), the category where existing suites are weakest. Coverage improvement is substantial (median $\Delta\text{BC} = +0.302$, large effect size).

The overall interpretation is that intermediate representations can serve as a useful control mechanism for LLM-based test generation within this domain and dataset. The most consistent benefit is behavioral coverage improvement and output stability relative to direct generation. Executability adds targeted advantages in traceability and analysis reuse value. Augmentation appears to outperform replacement as an adoption pattern under existing industrial test assets, provided that reference-selection robustness and instrument-reliability monitoring are treated as first-class operational concerns.

7

Conclusion

This chapter summarizes the findings, restates the contributions of this work, and outlines directions for future research.

7.1 Summary of Findings

We investigated whether intermediate representations improve LLM-based test generation for automotive system-level verification. Three research questions were addressed through a controlled within-subjects comparison on 31 industrial requirements from Volvo Group, using a unified four-path generation framework with shared discovery infrastructure, a single LLM family, and identical output DSL.

RQ1: Does using an intermediate representation improve LLM-based test generation from natural-language requirement compared to direct generation? Yes. The pooled comparison of IR-guided paths (B and C) against direct generation (Path A) showed statistically significant improvements in both behavioral completeness (BC, $p_{\text{adj}} = 0.0024$) and requirement traceability (RT, $p_{\text{adj}} = 0.0040$) after Bonferroni correction. Signal correctness and value correctness, already high for direct generation (SC ≥ 0.88, VC ≥ 0.98), did not change significantly with IR use. The disaggregated analysis confirmed that both IR types independently improve BC over direct generation. GT-SC triangulation against human-authored tests corroborated the trend, with Path C achieving the highest mean ground-truth scenario coverage (0.617 vs. 0.500 for Path A). Hallucination rate decreased from 0.020 (Path A) to 0.005 (Path B) and 0.000 (Path C in this sample).

RQ2: Does an executable IR produce better test designs than a structured IR? Partially. Path C (executable IR) significantly outperformed Path B (structured IR) on requirement traceability ($p_{\text{adj}} = 0.0135$), but the two paths were statistically indistinguishable on behavioral completeness, signal correctness, and value correctness. Both IR types reduced output variance relative to direct generation (BC SD: 0.068 for Path A vs. 0.047/0.046 for Paths B/C; SC SD: 0.030 vs. 0.008/0.006), indicating that structuring the generation process—regardless of executability—is a stability lever. The executable IR's primary measurable advantage is in traceability rather than coverage breadth.

RQ3: Can an executable IR support gap analysis of existing human-authored tests? Yes. The report-only enhancement pipeline identified gaps with 87.5% precision and 63.2% recall against an LLM-as-judge second rater. The candidate test preserved 94.7% of existing human-authored behaviors. Coverage improvement was substantial and significant: median $\Delta\text{BC} = +0.302$ ($p = 6.5 \times 10^{-5}$, Cliff's $\delta = +0.587$, large effect). Of the 30 paired comparisons, 22 showed positive improvement, 5 showed no change, and 3 showed a decrease (all three occurring on requirements where the existing test already had $\text{BC} \geq 0.75$). The category analysis revealed that most identified gaps (62.5%) were fault-condition behaviors, aligning with the practical observation that existing industrial suites tend to be stronger on nominal pathways than on fault-mode breadth.

7.2 Contributions

We make four contributions to the field of LLM-based test generation:

1. **A unified four-path framework** that isolates the effect of IR choice while holding constant the discovery context, generator family, and output DSL. This controlled design enables direct attribution of quality differences to the IR type rather than to confounding pipeline factors.

2. **A controlled empirical comparison** of no IR, structured IR, and executable IR on the same 31-requirement industrial dataset, showing that among the two IR types studied, structured intermediate decomposition accounts for the larger share of the measurable benefit, while executability adds a targeted traceability advantage.

3. **A requirement-centric evaluation design** combining four primary generation-quality metrics with a secondary triangulation metric against human-authored tests. The dual-oracle approach avoids the validity problem of measuring generated tests solely against potentially incomplete human-authored references.

4. **An IR-guided augmentation mode** for identifying requirement-coverage gaps in existing human-authored tests while preserving their content. The report-only design respects industrial workflows where existing tests are organizationally trusted and cannot be replaced wholesale.

7.3 Future Work

Several directions for future research emerge from this work:

- **Dynamic execution evaluation.** Execute generated tests on SIL and HIL rigs to measure pass rates, fault-detection effectiveness, and timing correctness. This would close the gap between the static quality metrics used here and the ultimate measure of test utility in a production verification environment.

- **Cross-domain evaluation.** Extend the evaluation to additional functional domains (e.g., powertrain, chassis, advanced driver assistance) and additional vehicle platforms to assess whether the IR benefit generalizes beyond the CCIOM body/cabin electronics domain.
- **Multi-model comparison.** Replicate the experiment with alternative LLM families (e.g., GPT-4o, Gemini, open-weight models such as Llama) to determine whether the IR benefit is model-specific or model-agnostic.
- **Retry-rate instrumentation.** Instrument the Path C orchestrator to log first-attempt xSpec validation success versus retry counts, enabling cost–benefit analysis of the executable IR's validation loop and informing decisions about when the additional pipeline complexity is justified.
- **Merged-output enhancement.** Extend the Path D pipeline from report-only mode to automatically produce merged test scripts that incorporate identified gaps into the existing human-authored test, with human review of the merge before deployment.
- **Evaluator instrument improvement.** Extend the manual validation to additional instruments (RT classification, discovery recall), potentially through fine-tuned classification models, expanded discovery agent coverage, or hybrid human–LLM evaluation workflows.
- **Longitudinal deployment study.** Deploy the augmentation pipeline in a production setting and measure its effect on engineering effort, test-suite coverage evolution, and defect escape rates over multiple release cycles.

7.4 Closing Remarks

This work provides evidence that intermediate representations can serve as a useful control mechanism for LLM-based test generation in an industrial automotive context. The strongest and most consistent benefit—improved behavioral completeness and output stability—emerges from the act of structuring the LLM's reasoning through either of the two intermediate representations studied, rather than from executability alone. For organizations with existing test assets, the augmentation pathway offers the most immediately applicable mode: it leverages the IR framework to identify coverage gaps without requiring wholesale replacement of trusted, human-authored tests. As LLM capabilities continue to evolve, the framework and evaluation design introduced here may provide a useful basis for measuring whether new models or prompting strategies close the remaining quality gaps.

Bibliography

[1] *Road vehicles – Functional safety*, International Organization for Standardization Std. ISO 26 262, 2018.

[2] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007.

[3] S. Kriebel, M. Markthaler, K. S. Salman, T. Greifenberg, S. Hillemacher, B. Rumpe, C. Schulze, A. Wortmann, P. Orth, and J. Richenhagen, "Improving model-based testing in automotive software engineering," in *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP '18)*. New York, NY, USA: Association for Computing Machinery, 2018, pp. 172–180. [Online]. Available: https://doi.org/10.1145/3183519.3183533

[4] A. M. Dakhel, A. Nikanjam, V. Majdinasab, F. Khomh, and M. C. Desmarais, "Effective test generation using pre-trained large language models and mutation testing," *Information and Software Technology*, vol. 171, p. 107468, 2024. [Online]. Available: https://doi.org/10.1016/j.infsof.2024.107468

[5] Z. Yuan, Y. Lou, M. Liu, S. Ding, K. Wang, Y. Chen, and X. Peng, "Evaluating and improving ChatGPT for unit test generation," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1703–1726, 2024. [Online]. Available: https://dl.acm.org/doi/10.1145/3660783

[6] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Liu, H. Chen, J. Yang, Z. Zhang, J. Guo *et al.*, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, 2024. [Online]. Available: https://doi.org/10.1007/s11704-024-40231-1

[7] M. Konstantinou, R. Degiovanni, and M. Papadakis, "Do LLMs generate test oracles that capture the actual or the expected program behaviour?" *arXiv*, 2024. [Online]. Available: https://arxiv.org/abs/2410.21136

[8] A. Celik and Q. H. Mahmoud, "A review of large language models for automated test case generation," *Machine Learning and Knowledge Extraction*, vol. 7,

no. 3, p. 97, 2025. [Online]. Available: https://doi.org/10.3390/make7030097

[9] M. L. Siddiq, J. C. D. S. Santos, R. H. Tanvir, N. Ulfat, F. A. Rifat, and V. C. Lopes, "Using large language models to generate junit tests: An empirical study," in *Proc. 28th Int. Conf. Evaluation and Assessment in Software Engineering (EASE)*, Jun. 2024. [Online]. Available: https://doi.org/10.1145/3661167.3661216

[10] Z. Yang, C. Cui, T. Li, R. Huang, N. Niu, D. Towey, and S. Guo, "Llmcfg-tgen: Using llm-generated control flow graphs to automatically create test cases from use cases," *arXiv preprint arXiv:2512.06401*, Dec. 2025. [Online]. Available: https://arxiv.org/abs/2512.06401

[11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[12] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," *Advances in Neural Information Processing Systems*, vol. 35, 2022.

[13] OpenAI, "Gpt-4 technical report," 2023. [Online]. Available: https://arxiv.org/abs/2303.08774

[14] Anthropic, "Claude model card," 2024. [Online]. Available: https://www.anthropic.com/model-card

[15] R. Anil *et al.*, "Gemini: A family of highly capable multimodal models," in *Proceedings of the 41st International Conference on Machine Learning*, 2024, google DeepMind Technical Report.

[16] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.

[17] J. Wei, X. Wang, D. Schuurmans, M. Bosma *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24 824–24 837.

[18] S. Yao, D. Yang, B. Cui, K. Narasimhan, and M. Powell, "React: Synergizing reasoning and acting in language models," *arXiv preprint arXiv:2210.03629*, 2022.

[19] T. Schick, J. Dwivedi-Yu, B. Krause *et al.*, "Toolformer: Language models can teach themselves to use tools," *arXiv preprint arXiv:2302.04761*, 2023.

[20] Z. Lyu, S. Hu, L. Liu, H. Yang, and M. Li, "Canonical intermediate representation for LLM-based optimization problem formulation and code generation," *arXiv preprint arXiv:2602.02029*, 2026.

[21] A. Ahmad, M. Waseem, P. Liang, M. Fahmideh, M. S. Aktar, and T. Mikkonen, "Evaluating ChatGPT's proficiency in understanding microservice systems via intermediate representations," in *International Conference on Evaluation and Assessment in Software Engineering (EASE)*, 2025, discusses decoupling process phases through intermediate representations to improve contextual interpretation.

[22] M. Utting and B. Legeard, *Practical Model-Based Testing: A Tools Approach*. Morgan Kaufmann, 2007.

[23] E. Bringmann and A. Kramer, "Model-based testing of automotive systems," in *2008 International Conference on Software Testing, Verification, and Validation*. IEEE, 2008, pp. 485–493.

[24] C. en A. Tai, L. Golab, and A. Wong, "Intermediate representations for improved code translation with LLMs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[25] S. et al., "Exploring the capabilities of large language models for UML class diagrams from Java and Python," in *CEUR Workshop Proceedings*, 2024.

[26] S. Chen, Z. Li, C. Liu, and Z. J. Chen, "Generating SysML behavior models via large language models: An empirical study," in *Proceedings of the 16th International Conference on Internetware*, 2024. [Online]. Available: https://dl.acm.org/doi/10.1145/3755881.3755926

[27] L. Yu *et al.*, "SysMBench: A system model generation benchmark from natural language requirements," 2025. [Online]. Available: https://arxiv.org/abs/2508.03215

[28] L. Ferrero *et al.*, "An agent-based approach for the automatic generation of valid SysML v2 models in industrial contexts," *Computers in Industry*, 2025. [Online]. Available: https://dl.acm.org/doi/10.1016/j.compind.2025.104350

[29] S. Xu *et al.*, "SimuAgent: An LLM-based Simulink modeling assistant enhanced with reinforcement learning," 2025. [Online]. Available: https://arxiv.org/abs/2601.05187

[30] W. Ma *et al.*, "GBCV: A generated benchmark from control-flow structure for evaluating LLM test generation," *MDPI Applied Sciences*, 2025. [Online]. Available: https://doi.org/10.3390/app15094753

[31] M. L. Siddiq *et al.*, "ULT and PLT: Large-scale benchmarks for evaluation of LLM-based unit test generation," 2025. [Online]. Available: https://arxiv.org/abs/2508.00408

[32] P. Han *et al.*, "TriGen: A semantic-feedback collaborative LLM test case generation model," *IEEE Access*, vol. 13, 2025. [Online]. Available:

https://doi.org/10.1109/ACCESS.2025.3525123

[33] D. Liao *et al.*, "Navigating the labyrinth: Path-sensitive unit test generation with large language models," 2025. [Online]. Available: https://arxiv.org/abs/2509.23812

[34] R. Zrelli *et al.*, "Natural2CTL: A dataset for natural language requirements and their CTL formal equivalents," in *Proceedings of the International Working Conference on Requirements Engineering: Foundation for Software Quality (REFSQ)*, 2024.

[35] A. Thakur *et al.*, "CLEVER: A curated benchmark for formally verified code generation," 2025. [Online]. Available: https://arxiv.org/abs/2505.13938

[36] R. Pan *et al.*, "Requirements coverage-guided minimization for natural language test cases (RTM)," 2025. [Online]. Available: https://arxiv.org/abs/2505.20004

[37] S. Xia *et al.*, "Doc2Spec: Synthesizing formal programming specifications from natural language via grammar induction," 2026. [Online]. Available: https://arxiv.org/abs/2602.04892

[38] A. Neumann and T. Yarkoni, "Transitions: A lightweight, object-oriented finite state machine implementation in python," https://github.com/pytransitions/transitions, 2014.

[39] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[40] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2306.05685

[41] N. Cliff, "Dominance statistics: Ordinal analyses to answer ordinal questions," *Psychological Bulletin*, vol. 114, no. 3, pp. 494–509, 1993.

[42] J. Romano, J. D. Kromrey, J. Coraggio, and J. Skowronek, "Appropriate statistics for ordinal level data: Should we really be using t-test and Cohen's d for evaluating group differences on the NSSE and other surveys?" *Annual Meeting of the Florida Association of Institutional Research*, pp. 1–33, 2006.

[43] B. Danglot, O. L. Vera-Perez, Z. Yu, A. Zaidman, M. Monperrus, and B. Baudry, "A snowballing literature study on test amplification," *Journal of Systems and Software*, vol. 157, p. 110398, 2019. [Online]. Available: https://doi.org/10.1016/j.jss.2019.110398

[44] M. Tufano, S. K. Deng, N. Sundaresan, and A. Svyatkovskiy, "Methods2test:

A dataset of focal methods mapped to test cases," in *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*, 2022, pp. 299–303. [Online]. Available: https://doi.org/10.1145/3524842.3528009

[45] S. Wang, Y. Yu, R. Feldt, and D. Parthasarathy, "Automating a complete software test process using LLMs: An automotive case study," in *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE, 2025.

A

Appendix

A.1 Manual Validation Sample

Table A.1 lists the six requirements sampled for the manual validation study described in Section 4.6.4. Requirement handles were jointly agreed upon by the two raters before either opened a packet. Complexity tiers for the validation sample are assigned by the number of expected behaviors extracted by the evaluator agent (simple: 1–3; medium: 4–8; complex: ≥ 9), which serves as an operational proxy for the specification-length-based classification used in Section 4.2.3. Note that these two operationalizations may assign different tiers to the same requirement; their purpose is to ensure tier diversity within the validation sample rather than to replicate the dataset classification exactly. One Path C run per requirement was selected for the per-output protocols.

Table A.1: Validation sample: handles, tiers, and expected-behavior counts.

Handle	Requirement name	$\mathbf{n}_{\text{exp}}$	Tier
`x040000002CE1FCDE`	BatteryVoltageDisplayValue_SigDid	1	Simple
`x0400000028AA60CA`	Battery voltage presentation variants	6	Medium
`x0400000028B31956`	Battery Voltage low pass filter	6	Medium
`x0400000028C3F3F0`	Charger Connection for SESS	8	Medium
`x0400000028AAD449`	Battery charging voltage regulation	9	Complex
`x0400000028C3D7E0`	Battery status Unknown Warning	9	Complex

Table A.2 summarises the aggregate counts from the validation study.

Table A.2: Manual-validation audit summary.

Quantity	Value
Requirements sampled for manual validation	6
Canonical behaviors extracted by evaluator agent	39
Human-extracted expected behaviors (rater A / rater B)	38 / 35

A.2 Validation Procedure and Worked Example

This section documents the validation packet structure, a worked example showing one handle scored end-to-end, and the evaluation tooling used in the manual validation study (Section 4.6.4). Both raters applied the same scoring rubric defined in Section 4.6.4; metric formulas are not repeated here. After completing all rows, each rater re-scored a 10% random sample after a cooldown period; if absolute drift exceeded 0.10 on any metric, scoring was paused and rubric decisions were recalibrated before resuming. Full per-handle scoring workbooks for all six sampled requirements are retained in the project audit archive.

A.2.1 Validation Packet Structure

For each sampled requirement, the packet-builder script (Section A.2.3) assembled the following inputs per `(handle, path, run)` triple:

1. **Discovery JSON** — the evaluator agent's output containing requirement text, extracted signals (name, bus, direction, failure modes), applicable vehicle modes, and configurable parameters (DOIDs).

2. **Generated script** (`content.md`) — the PN-Script test case produced by the generator for that path and run.

3. **Blank scoring row** — a CSV row pre-populated with identifiers (`handle`, `path_label`, `run`, `tier`) and empty columns for the five metrics plus free-text evidence notes.

Raters were instructed to score each row using *only* the discovery JSON and the generated script. Reading the pipeline's own `evaluation.json` was prohibited to prevent circular evaluation.

A.2.2 Worked Example

This section traces one medium-complexity requirement through the full validation protocol for Rater A. The requirement (*Battery voltage presentation variants*) specifies a low-pass filter on a voltage signal with mode-dependent applicability and variant-specific raw-input selection. The evaluator agent extracted six expected behaviors ($|E_{\text{req}}| = 6$); the generated Path C script contained seven behavioral blocks ($|B_{\text{gen}}| = 7$).

Step 1: Human behavior extraction. Rater A independently identified six expected behaviors from the requirement text:

H1 IIR filter equation: $y(n) = k \cdot x(n) + (1-k) \cdot y(n-1)$.

H2 Filter gain k equals configurable DOID `P1NL1`.

H3 Raw-data initialization: first valid sample seeds filter.

H4 Raw-data fault recovery: invalid→valid resets filter.

H5 VehicleMode fault recovery: invalid→valid resets filter.

H6 Applicable vehicle modes: Parked, Living, Accessory, Pre Running, Cranking, Running.

Step 2: BC extraction check — agent-extracted behavior judgement. Each agent-extracted behavior was compared against the human-extracted set.

Table A.3: BC Phase 2 judgements for `x0400000028AA60CA` (worked example, Rater A).

ID	Judgement	Description (truncated)	Matched
eb1	correct	VehicleMode invalid→valid → filter reset	H5
eb2	correct	raw_data at start → seed filter with present value	H3
eb3	correct	raw_data invalid→valid → filter reset	H4
eb4	correct	$x(n) \rightarrow y(n) = k \cdot x(n) + (1-k) \cdot y(n-1)$	H1
eb5	correct	Applicable VehicleModes: Parked. . . Running	H6
eb6	correct	Filter gain k = `Voltage_gain_filter` (DOID `P1NL1`)	H2

All six agent-extracted behaviors were judged correct, yielding BC = 6/6 = 1.000.

A.2.3 Evaluation Tooling

The validation workflow was driven by two Python scripts, both operating independently of the pipeline's own `evaluation.json`. The **packet builder** (`build_packets.py`) locates the discovery JSON and generated `content.md` for each `(handle, path, run)` triple, derives a complexity tier, and emits a manifest CSV plus a blank scoring template with pre-populated identifiers. The **score aggregator** (`aggregate_scores.py`) computes per-handle–per-path means over runs, path-level summaries (mean and SD over handles), pooled IR-vs-baseline deltas, and Cliff's delta effect sizes, emitting four CSV files consumed by the thesis tables. Source code for both scripts is included in the project repository under `analysis/semantic_pilot/`.

A.3 RQ3 Per-Handle Enhancement Results

Table A.4: Per-handle enhancement results (RQ3).

Handle (last 8)	BC_{ex}	BC_{enh}	ΔBC	n_{exp}	Gaps	Pres.	Lost
28DE64D5	0.148	1.000	+0.852	—	—	—	—
28ACC5F2	0.083	0.917	+0.833	6	6	0	0
28AA55A6	0.167	1.000	+0.833	6	5	1	0
28C09A64	0.250	1.000	+0.750	6	6	0	0
28DE7EA6	0.107	0.786	+0.679	14	10	1	0
28C08158	0.000	0.636	+0.636	11	7	0	0
28AA60CA	0.250	0.833	+0.583	6	5	1	0
28C3D7E0	0.333	0.889	+0.556	9	5	3	0
28C4070E	0.278	0.833	+0.556	9	6	3	0
28C09A6D	0.273	0.818	+0.546	11	9	0	0
28DE2F4F	0.500	1.000	+0.500	5	5	0	0
28DE7715	0.286	0.786	+0.500	7	5	2	0
28B31956	0.583	1.000	+0.417	6	3	3	0
28ACDABB	0.538	0.885	+0.346	13	6	7	0
28DE2F8E	0.500	0.818	+0.318	11	4	5	0
28AAD4A1	0.714	1.000	+0.286	7	2	5	0
28DE37D4	0.750	1.000	+0.250	8	2	6	0
28AAD449	0.778	1.000	+0.222	9	2	7	0
28C3F3F0	0.438	0.625	+0.188	8	2	3	0
28DE770D	0.792	0.958	+0.167	12	3	9	0
28C1CC1A	0.833	1.000	+0.167	3	1	2	0
28E81782	0.571	0.595	+0.024	21	1	12	0
283A0D03	1.000	1.000	0.000	4	0	4	0
286323D2	1.000	1.000	0.000	2	0	2	0
28B2B6D5	1.000	1.000	0.000	8	0	8	0
28DE2F27	1.000	1.000	0.000	2	0	2	0
2CE1FCDE	1.000	1.000	0.000	1	0	1	0
2B9A48C9	0.750	0.700	−0.050	10	1	6	2
28C434F2	0.750	0.625	−0.125	4	0	3	0
28637AB3	1.000	0.733	−0.267	15	0	11	4
28DE772D	—	1.000	—	—	—	—	—

Table A.4 reports per-handle ΔBC values and gap-analysis counts for all 31 requirements evaluated in RQ3 (30 with complete ΔBC data). BC_{ex} is the behavioral completeness of the human-authored test; BC_{enh} is the score of the best Path C run

used as the enhancement reference. Handles are sorted by ΔBC descending. One handle (`28DE64D5`) has a transient evaluation failure that prevents gap counting; one handle (`28DE772D`) has no existing-test evaluation and is excluded from the paired analysis. One handle (`28C434F2`) shows ΔBC = −0.125 despite Lost= 0; this is consistent with the partial-credit BC formula, where behaviors scored as partially covered can lower BC_{enh} without any behavior being fully lost.

A.4 JSON IR Schema

The structured JSON intermediate representation (`reasoning.json`, Path B) is defined by the generator agent instructions rather than a standalone JSON Schema file; validation is performed by checking that the output is valid JSON and contains all required top-level keys. The top-level object contains seven sections: `requirement` (handle, name, architecture, ECU, description), `signal_analysis` (input/output/internal signals with bus type, failure modes, and possible values), `parameters` (DOID entries), `behaviors` (see Listing A.1), `vehicle_modes` (applicable and non-applicable mode lists), `coverage_plan` (behavior count by category), and `limitations` (flags for capabilities absent in the structured IR: no edge detection, no timing validation, no state machine, no execution).

The per-behavior entry format is shown below.

```
{
  "id": "b1",
  "description": "Natural language description of the behavior
     ",
  "category": "nominal|fault|boundary|timing|mode_interaction|
     invalid_inputs|state_persistence",
  "trigger_type": "edge|level",
  "condition": {
    "signal": "input_signal_name",
    "operator": "==|!=|changes_to|in_range",
    "value": "trigger value or range",
    "trigger_note": "explanation of when this triggers"
  },
  "preconditions": [
    { "parameter": "DOID or signal", "value": "required state"
       }
  ],
  "expected_outputs": [
    { "signal": "output_signal_name", "value": "expected value
       ",
      "access_method": "bus|DID" }
  ],
  "vehicle_modes": [2,3,4,5,6],
  "rationale": "Why this behavior needs testing"
}
```

Listing A.1: Behavior entry in `reasoning.json`.

Each behavior entry records the trigger condition, expected output, applicable ve-

hicle modes, and a rationale field that traces the behavior back to a specific requirement clause. The behavior categories are shared with the xSpec IR (Appendix A.5), enabling direct comparison of behavioral coverage across Paths B and C.

A.5 xSpec Framework

The executable specification framework consists of four core components, all contained in a single Python module. The framework uses the `transitions` library [38] for state-machine infrastructure. Listing A.2 shows the implementation.

```
from transitions import Machine
from dataclasses import dataclass, field
from typing import Any, Callable, List, Optional
import time

class VehicleMode:
    """Vehicle␣mode␣constants␣matching␣SE-tool/PN␣Script␣
       values."""
    PARKED = 1
    LIVING = 2
    ACCESSORY = 3
    PRE_RUNNING = 4
    CRANKING = 5
    RUNNING = 6

    @classmethod
    def name(cls, value: int) -> str:
        ...  # returns string label for each integer mode
           constant


@dataclass
class Behavior:
    """A␣single␣behavior␣extracted␣from␣a␣requirement."""
    id: str
    description: str
    category: str = "nominal_behavior"
    preconditions: List[str] = field(default_factory=list)
    condition: str = ""
    expected_output: str = ""
    vehicle_modes: List[int] = field(default_factory=list)


class BaseXSpec:
    """
␣␣␣␣Base␣class␣for␣executable␣specifications.
␣␣␣␣Subclasses␣define␣states,␣transitions,␣signals,␣and␣
   behaviors.
␣␣␣␣"""
    states: List[str] = ["initial"]
    requirement_handle: str = ""
    requirement_name: str = ""
    vehicle_modes: List[int] = []
    state: str
```

```
    def __init__(self, initial_state: str = "initial"):
        self.machine = Machine(
            model=self, states=self.states, initial=
                initial_state)
        self._behaviors: List[Behavior] = []
        self._edge_tracking: dict[str, Any] = {}
        self._timers: dict[str, Optional[float]] = {}

    def _setup_transitions(self) -> None: pass
    def _setup_signals(self) -> None: pass
    def _register_behaviors(self) -> None: pass
    def update(self) -> None: pass

    def register_behavior(self, b: Behavior) -> None:
        self._behaviors.append(b)

    def get_behaviors(self) -> List[Behavior]:
        return self._behaviors

    # --- Edge Detection ---
    def _init_edge_tracking(self, signal: str, val: Any):
        self._edge_tracking[f"_prev_{signal}"] = val

    def _update_edge_tracking(self, signal: str):
        self._edge_tracking[f"_prev_{signal}"] = getattr(
            self, signal, None)

    def _check_rising_edge(self, signal: str) -> bool:
        prev = self._edge_tracking.get(f"_prev_{signal}",
            False)
        return not prev and getattr(self, signal, False)

    def _check_falling_edge(self, signal: str) -> bool:
        prev = self._edge_tracking.get(f"_prev_{signal}", True
            )
        return prev and not getattr(self, signal, True)

    def _check_value_change(self, signal: str, to: Any) ->
        bool:
        prev = self._edge_tracking.get(f"_prev_{signal}")
        return prev != to and getattr(self, signal, None) ==
            to

    # --- Timers ---
    # Timer helpers wrap time.time() for debounce and deadline
        checks
    def _start_timer(self, name: str): ...
    def _stop_timer(self, name: str): ...
    def _check_timeout(self, name: str, seconds: float) ->
        bool: ...


class ScenarioRunner:
    """Runner for executing test scenarios against an xSpec.""
        "
```

```
    def __init__(self, spec_class: type):
        self.spec_class = spec_class
        self._scenarios: List[tuple[str, Callable]] = []

    def scenario(self, name: str):
        """Decorator␣to␣register␣a␣test␣scenario."""
        def decorator(func: Callable):
            self._scenarios.append((name, func))
            return func
        return decorator

    def run_all(self) -> bool:
        # Instantiates a fresh spec per scenario; catches
            AssertionError
        # and exceptions; returns overall pass/fail bool
        ...
```

Listing A.2: xSpec framework: `VehicleMode`, `Behavior`, `BaseXSpec`, and `ScenarioRunner`.